iResolve Module Spec — Placement Intake (Flat Header Files, Phase 1)

> **Audience: Codex.** This document is the full brief: design, data model, API, a very detailed UI spec, and reference code.
> Read **section 0** first. It says which code is tested and can be dropped in, and which code must be adapted to the iResolve codebase.

---

0. READ FIRST — How to use the code in this document

The author of this spec does **not** have access to the iResolve codebase. The code was written outside it. Every code block in Appendix B carries one of three labels:

Label	Meaning	What Codex must do
🟢 **DROP-IN · TESTED**	Plain Python, no Django or iResolve dependency. Run against synthetic placement files (pipe and comma delimited, trailer rows, blank or duplicate accounts, bad dates, two date columns, headerless and fixed-width files, Excel template round trip).	Copy into the app folder unchanged. Do not rewrite the logic. Add unit tests around it (section 15).
🟡 **ADAPT · NOT RUN**	Standard Django / DRF / Celery code. It compiles (`py_compile`) but was **never run**, because Django was not available. It assumes generic conventions.	Use as the reference implementation. Fit it into iResolve's real structure: auth, permission system, settings, existing Celery app, URL layout, migrations, logging. Keep the behaviour, endpoint shapes and validation rules.
🟠 **REFERENCE UI · NOT COMPILED**	React components for the hardest interactions only (column picker, client-by-folder table, template import report, run polling). Never compiled or	Rebuild with iResolve's own component library and styles. Keep the state shape, validation, keyboard behaviour and wording exactly as written. Section 10 is

	rendered.	the UI source of truth, not these files.

Assumptions to confirm or correct before building (each one is marked in the code):

1. **Backend is Django + Django REST Framework + Celery + Postgres.** If iResolve uses plain Django views, convert the `@api_view` functions to views returning `JsonResponse` with the same URLs, payloads and status codes.
2. Permissions are checked in `permissions.py::user_has_perm()`. **Wire this to iResolve's module-permission system.** Never rely on hiding buttons.
3. The Artiva lookup (`artiva_check.py`) needs real table and column names per prod and the existing pyodbc connection helper. Until then it returns `{}` and everything still works, just with less auto-mapping confidence.
4. The frontend is React (Vite) using iResolve's existing component library. The API client in `api.js` must be replaced with iResolve's HTTP helper (auth, CSRF, base URL).
5. `openpyxl` is required (Excel template). Confirm it is in the deployment bundle.

Anything in this document that conflicts with how iResolve already does something (naming, folder layout, auth): follow iResolve and keep this module's behaviour.

---

1. Goal

Clients send placement files (new accounts to load into Artiva) into segment folders on the shared drive. For every file, extract:

Output field	Source
`file_name` , `file_path`	file system
`account_seq`	**Account serial number**: position of the account in the placement file (1st account = 1). Counts every data record, rejected ones too, so "account #10" is always the 10th account in the file.
`line_number`	physical line in the file (header is line 1), for opening the file and jumping to the row
`account_number`	column in the file
`chargeoff_date`	column in the file

`client_name`	client assigned to the folder (most files), or a column in the file
`client_ref`	column in the file (optional)

Write the results to **Postgres** (`pi_accounts` , joined to `pi_files`) and export them to **CSV or pipe-delimited TXT**. **Every run also writes a detailed log file** (section 8.2), downloadable next to every export button.

Scope

- **Phase 1 (this spec):** flat files with a header row.
- **Phase 2:** multi-record / fixed-width files. These are detected and parked now (section 11).

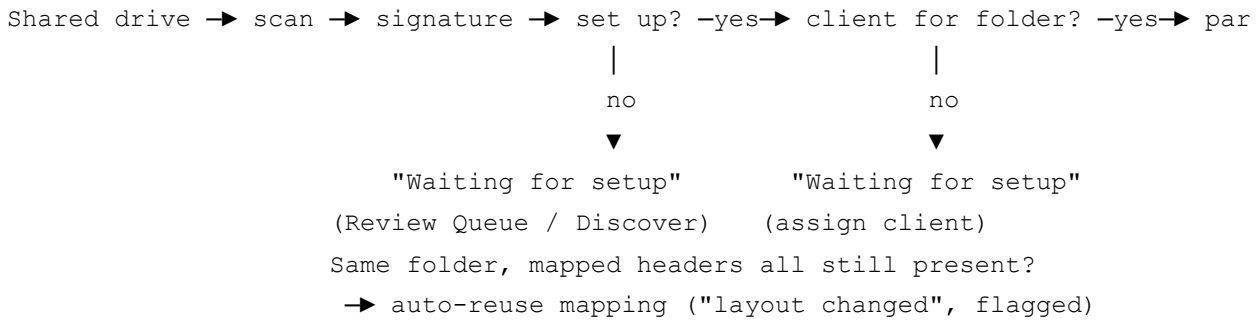
Scale: about 10 segments, hundreds of portfolios, about 50 distinct layouts, 4 prods (tmobile, attprod, crmprod, and one more).

---

2. Core Ideas

1. **Set up file types, not portfolios.** A *file type* is identified by its **header signature**: a hash of the normalized header row. Many portfolios share one file type. New files with a known signature process with no setup.
2. **Mappings live in data, not code.** The parser is generic and reads a versioned config from Postgres. A new file type never needs a code change or a rebuild.
3. **The machine suggests, a human confirms.** Discovery pre-fills each file type's mapping (header synonyms, value profiling, Artiva match) with a confidence and a plain-language reason. The user confirms or corrects the columns and assigns the client.
4. **Two ways to set up file types, same result:**
   - **Option 1, Discover (frontend):** pick folders → scan → for each file type, pick the columns and assign clients → save.
   - **Option 2, Excel template:** download a template (blank, or pre-filled from a scan) → fill one row per file type per folder → upload → iResolve checks every row against a real file → fix in place → save.
5. **It learns.** Every confirmed header name is added to the synonym list, so the next similar file type pre-fills better.
6. **Nothing is lost silently.** Every file ends in a visible state: Processed, Partial, Waiting for setup, Parked (Phase 2), or Failed, with a reason.
7. **Every run is traceable.** Each scan, processing run and reprocess writes its own detailed

log file. Every extracted account carries its serial number in the source file, so any row in the export can be traced back to its exact position in the placement file.



---

3. Vocabulary — code vs. UI (use the UI words on screen)

In code / DB	On screen	Meaning
`Format` (`pi_formats`)	**File type**	One distinct header layout
`signature`	*(never shown; Advanced drawer only)*	Hash of normalized headers
`FormatVersion`	**Version**	Immutable snapshot of a mapping
`field_maps`	**Columns**	Which column is which output field
`FormatClient`	**Client by folder**	Which client a folder (or a file-name pattern in it) belongs to
`status=needs_review`	**Needs setup**	No approved mapping yet
`PlacementFile.status=needs_review`	**Waiting**	File held until its type or client is set up
`headerless`	**Parked**	Fixed-width / no header; Phase 2
`partial`	**Done with errors**	Some rows rejected

`auto_drift`	**Layout changed (auto)**	New layout reused an old mapping automatically
`run`	**Scan** (discovery) / **Processing run**	A background job

Never show the words *signature*, *field_maps*, *JSON*, *regex*, *format_version* or *Celery* to users.

---

4. Architecture

- **Django app:** `placement_intake` (models, services, DRF views, URLs, Celery tasks, management command).
- **Mounted at:** `path("api/placement-intake/", include("placement_intake.urls"))`.
- **Jobs (Celery):** discovery scan, processing run (scheduled every N minutes plus a manual button), reprocessing of waiting files after setup.
- **DB:** Postgres (the website prod DB). `JSONField` maps to `jsonb`.
- **Artiva:** read-only pyodbc to Caché/IRIS, sampled, optional (section 7.3).
- **Frontend:** React pages under `/modules/placement-intake/*`.
- **Server constraints:**
  - Windows Server, HIPAA, no new software installs
  - shares reached via UNC paths under the service account
  - stdlib plus packages already bundled; `openpyxl` needs confirming

```
placement_intake/
  __init__.py
  apps.py
  common.py          🟢 helpers: normalize, signature, dates, masking, hashing
  discovery.py       🟢 scan folders, sample files, group by signature
  automap.py         🟢 suggest columns + confidence + reasons
  parser.py          🟢 config-driven parser + row validation
  template_io.py     🟢 Excel template build / read / validate / errors workbook
  run_log.py         🟢 one detailed .log file per run (+ retention cleanup, tail
  cli_discover.py    🟢 standalone dry run (no Django)
  artiva_check.py    🟡 optional Artiva ground-truth signal (fill table names)
  models.py          🟡 Django models
  services.py        🟡 all DB logic (views and tasks call only this)
  tasks.py           🟡 Celery tasks
```

```
  permissions.py      🟡 permission classes (wire to iResolve)
  views.py            🟡 DRF endpoints
  urls.py             🟡 routes
  management/commands/pi_seed.py  🟡 seeds synonyms + default settings
frontend/src/modules/placement-intake/
  api.js                    🟠 replace with iResolve HTTP helper
  useRunPolling.js          🟠
  FileTypeSetup.jsx         🟠 column picker + client by folder (the hardest scre
  TemplateImportReport.jsx  🟠
  ...other pages built from section 10
```

---

5. Data Model

Full definitions are in `models.py` (Appendix B). Tables:

Table	Purpose
`pi_settings`	Key/value: `roots` (watched folders), `export_dir`, `export_format`, `auto_approve` (default false), `schedule_minutes`, `log_dir` (default `<BASE_DIR>/logs/placement_intake`), `log_retention_days` (default 90)
`pi_clients`	Client code, name and prod. Feeds every client dropdown. Loaded by CSV or synced from Artiva.
`pi_segment_defaults`	Folder prefix → prod, segment, optional date format
`pi_header_synonyms`	Header aliases per field (`seed` or `learned`)
`pi_formats`	One row per file type (signature, headers, delimiter, status, latest suggestion, example file)
`pi_format_versions`	Immutable mapping snapshots: `parse_config` + `field_maps` + who approved and how
`pi_format_folders`	Folders where each file type was seen, with counts
`pi_format_clients`	Client per (file type, folder, optional file-name pattern). Most specific match wins.
`pi_runs`	Scans and processing runs, with live progress, stats, `export_path` and `log_path`

`pi_discovery_items` / `pi_discovery_files`	What one scan found (drives wizard step 2 and the Parked list)
`pi_files`	One row per processed file (hash for idempotency, status, counts, client, version used)
`pi_accounts`	Extracted rows, with `account_seq` (serial in file) and `row_num` (line number)
`pi_row_errors`	Rejected rows (`account_seq` , line number, field, error code; never the value)
`pi_template_imports`	Uploaded template + dry-run results, until committed

5.1 `parse_config` and `field_maps` (JSON on `pi_format_versions`)

```
// parse_config
{"delimiter": "|", "quotechar": "\"", "encoding": "utf-8-sig", "header_row": 0,
 "skip_footer_rows": 1, "footer_marker_regex": "^(TRL|TRAILER|TOTAL)",
 "date_formats": ["%m/%d/%Y"]}

// field_maps
{"account_number": {"source": "header", "header": "Acct No", "index_fallback": 0}
 "chargeoff_date": {"source": "header", "header": "C/O Date"},
 "client_name":    {"source": "assigned_client"},
 "client_ref":     {"source": "header", "header": "Client Ref"}}
```

Other `source` values: `index` (`{"index": 3}`), `constant` (`{"value": "..."}`), `filename_regex` (`{"pattern": "...", "group": 1}`), and later `position` for Phase 2.

5.2 Error codes (stored) → messages (shown)

Code	Shown as
`blank_account`	Account number is blank
`blank_date`	Charge-off date is blank
`bad_date`	Charge-off date can't be read with this file type's date format
`future_date`	Charge-off date is in the future

`dup_in_file`	Account number appears more than once in this file (first one kept)
`col_count_mismatch`	Row has a different number of columns than the header

---

6. Option 1 — Discover from the frontend

Engine: `discovery.scan()` + `automap.guess_mapping()`, run by the Celery task `discover_task` and stored by `services.run_discovery()`.

Flow (UI detail in section 10.6):

1. **Choose folders** from a tree of the watched roots and start the scan.
2. **Scan results:** file types found (New / Already set up / Layout changed) plus a Parked list.
3. **Set up each file type:** pick columns (pre-filled, with confidence and reason), date format with a live preview, client source, client by folder (with split by file-name pattern).
4. **Done:** summary, with waiting files processed automatically.

Rules:

- The latest file of each signature is profiled (500 rows by default).
- Masked samples are produced on request and never stored. Only the chosen date column's raw values go to the browser, for the date preview.
- Saving creates a new version (`approval="human"`) and client rows (`source="wizard"`), learns the header names, and queues waiting files.
- Re-scanning the same folders only puts *new* or *changed* types in front of the user.

7. Auto-mapper (suggestions)

Implemented in `automap.py` 🟢. Per (field, column):

- **Header score:** 1.0 for an exact synonym; otherwise the best fuzzy ratio if ≥ 0.75; else 0.
- **Value score:**

Field	Value score
account_number	uniqueness, ID-like pattern, consistent length; penalized if values look like dates
	share of values that are valid past dates; the **earliest** date column is

chargeoff_date	favoured (charge-off comes before placement and open dates)
client_ref	uniqueness and ID-like pattern
client_name	at most 5 distinct text values

- **Artiva score (optional, 7.3):** share of sampled values found in Artiva.
- **Combined:** `0.6·header + 0.4·value`, or with Artiva `0.35·header + 0.25·value + 0.40·artiva`.
- **Confidence:**
  - **HIGH:** score ≥ 0.75 and ≥ 0.15 above the runner-up, or an account column with ≥ 90% Artiva match.
  - **MEDIUM:** score ≥ 0.5.
  - **LOW:** otherwise.
- **Assignment:** greedy by score; a column is used at most once.
- **Date guard:** values outside the years 1900–2100 do not count as dates, so numeric IDs like `7000101` are not mistaken for `%Y%m%d`.

7.3 Artiva ground truth 🟡

`artiva_check.candidate_rates()` samples up to 300 values from the top candidate columns and looks them up in Artiva (IN-lists of 100, read-only).

- **Account:** share found.
- **Charge-off date:** for matched accounts, the share whose date equals Artiva's.

Fill `ARTIVA_LOOKUP` per prod. Values are never logged; only rates are.

7.4 Auto-approve

A Settings toggle, **OFF by default**. When on, a new file type with HIGH confidence for both required fields, in folders that already have a client assigned, is approved without a human.

8. Processing

`services.process_new_files()` 🟡, run by Celery on a schedule and by the **Process new files now** button. For each flat file under the roots:

1. **Duplicate check.** If the file's sha256 is already in `pi_files`, skip it.
2. **Read the header.** Headerless or fixed-width files are marked **Parked**.

3. **Find the file type.** If the signature is not approved, try **drift**: an approved type in the same folder whose mapped headers all exist in the new header. If found, clone it (`auto_drift`) and flag it. Otherwise create a *Needs setup* type and mark the file **Waiting**.
4. **Find the client.** Use the most specific `FormatClient` match. If there is none, the file is **Waiting** ("No client assigned").
5. **Parse and validate.** Bulk-insert valid rows into `pi_accounts` (batches of 5,000) and errors into `pi_row_errors` . The file is **Processed** or **Done with errors**.
   - A mapped header missing from the file → **Failed**, with the message naming the column. Columns are **never** silently shifted.
6. **Export.** The run's export file goes to `export_dir` :
   `file_name, file_path, account_seq, line_number, account_number, chargeoff_date (YYYY-MM-DD), client_name, client_ref` .
   - `account_seq` = the account's serial number in the placement file. Gaps in the export (e.g. 9, 11) mean account #10 was rejected; the log and the Files error drawer show why.
7. **Log.** One line per file plus detail lines (section 8.2).

After a file type is set up (wizard or template), its waiting files are reprocessed automatically, in their own run with their own log.

8.1 Export file example

```
file_name,file_path,account_seq,line_number,account_number,chargeoff_date,client_
ATT_PLC_20261021.txt,\\fs01\Placements\Telecom\ATT\ATT_PLC_20261021.txt,1,2,10000
ATT_PLC_20261021.txt,\\fs01\Placements\Telecom\ATT\ATT_PLC_20261021.txt,2,3,10000
```

8.2 Run logs (`run_log.py` 🟢, wired in `services.run_logged()` 🟡)

Every run writes a log: discovery scans, processing runs (scheduled and manual), reprocessing after setup, and single-file reprocess.

File and storage

- **Name:** `pi_run000041_process_20261022_103000.log`
- **Folder:** `log_dir` .
- **Path stored on the run:** `pi_runs.log_path` , saved **when the run starts**, so the log can be downloaded while the run is going (the file is line-buffered) and after a crash.
- **Retention:** logs older than `log_retention_days` (default 90; 0 = keep forever) are

deleted at the end of each processing run, and the deletion itself is logged.

Content rule: as detailed as possible, but **never account values** (no account numbers, names, dates from rows). Only paths, headers, counts, codes, settings, timings, and account serial and line numbers.

What each log contains

Part	Details
Header	run id, mode, who started it (or `schedule`), host, process id, Python version, run parameters, settings in effect (roots, export dir and format, auto-approve, retention)
Folder problems	every root or sub-folder that is missing or not accessible: `ERROR [FOLDER] <path> - <reason>`
Per file (processing)	a section line with the file name, then: full path · size · modified time · sha256 (short) · prod/segment (or "no segment default") · delimiter, column count, signature · all headers · file type name + version + how/by whom/when it was set up · column mapping · date formats, footer rows, encoding · client and which rule matched (pattern or whole folder) · insert progress every 50,000 accounts · records / loaded / rejected · for each rejection code, the first 10 accounts as `#serial (line N)` · time taken and rows/s · the result line
Result line per file	`[PROCESSED] / [PARTIAL] / [FAILED] / [NEEDS_REVIEW] / [HEADERLESS] <path> file_type=… client=… rows_ok=… rows_error=… errors=code:n,… - message` (WARN for partial / waiting / parked, ERROR for failed)
Skips	every file not processed: `[ALREADY DONE] <path> - same content as file #123`, `[SKIP] <path> - not a flat file type (.xlsx)`, `[UNREADABLE] <path> - <OS error>`
Layout change	`WARN LAYOUT CHANGED: reused mapping of '<type>' v3 … -> new file type '<type> (changed layout)'`
Per file type (discovery)	name and state (new / existing / changed) · files, first and last seen, delimiter, columns, signature · all headers · each folder with its file count · latest file profiled · each suggested column with confidence and reason · missing suggestions as WARN · whether Artiva was checked
	export file path and rows written · summary counts · logs pruned · `=== Run #41 FINISHED in 3m 12s · 4 warnings · 1 errors ===`. If the run

Footer	crashes: `ERROR Run stopped with an error: …`, the last traceback lines, and `FAILED.`

Example (processing run):

```
2026-10-22 10:30:00 INFO  === Run #41 (process) started by schedule ===
2026-10-22 10:30:00 INFO  Environment: host=CRM-IRES01 · pid=4120 · python=3.12.4
2026-10-22 10:30:00 INFO  Run parameters: roots=\\fs01\Placements · since_days=No
2026-10-22 10:30:00 INFO  Settings: roots=\\fs01\Placements · export_dir=\\fs01\E
2026-10-22 10:30:00 INFO  Watched folders: \\fs01\Placements
2026-10-22 10:30:01 ERROR [FOLDER] \\fs01\Placements\Healthcare\HCA_OLD - [WinErr
2026-10-22 10:30:01 INFO      [ALREADY DONE] \\fs01\Placements\Telecom\ATT\ATT_PL
2026-10-22 10:30:01 INFO  --- ATT_PLC_20261021.txt ------------------------------
2026-10-22 10:30:01 INFO      path=\\fs01\Placements\Telecom\ATT\ATT_PLC_20261021
2026-10-22 10:30:01 INFO      size=1,204,113 bytes · modified=2026-10-21 23:14:02
2026-10-22 10:30:01 INFO      prod=attprod · segment=Telecom
2026-10-22 10:30:01 INFO      layout: delimiter='|' · columns=6 · signature=4be1d
2026-10-22 10:30:01 INFO      headers: ACCT_ID | CO_DT | OPEN_DT | CRED | CUST_RE
2026-10-22 10:30:01 INFO      file type: 'ATT – ATT_PLC_*' v2 (set up via human b
2026-10-22 10:30:01 INFO      columns: account_number=ACCT_ID · chargeoff_date=CO
2026-10-22 10:30:01 INFO      config: date_formats=['%m/%d/%Y'] · skip_footer_row
2026-10-22 10:30:01 INFO      client: ATTW - ATT Wireless (matched whole folder ·
2026-10-22 10:30:03 INFO      records=10,000 · loaded=9,980 · rejected=20
2026-10-22 10:30:03 INFO      rejected bad_date: first at account #6 (line 7), #4
2026-10-22 10:30:03 INFO      rejected blank_account: first at account #5 (line 6
2026-10-22 10:30:03 INFO      took 1.9s · 5,263 rows/s
2026-10-22 10:30:03 WARN  [PARTIAL] \\fs01\Placements\Telecom\ATT\ATT_PLC_2026102
...
2026-10-22 10:33:11 INFO  Extract written: \\fs01\Extracts\placement_extract_run4
2026-10-22 10:33:11 INFO  Summary: seen=44 · skipped_known=2 · processed=39 · par
2026-10-22 10:33:12 INFO  === Run #41 FINISHED in 3m 12s · 4 warnings · 2 errors
```

9. Option 2 — Excel template

Implemented in `template_io.py` 🟢 and `services.template_*` 🟡. UI detail in section 10.7.

- **Download blank** or **pre-filled from a scan.** Pre-filled has folder, file pattern, delimiter and suggested headers already in, and lists all headers in Notes.
- **Sheets:**
  - **Instructions:** how to fill it, plus an example row.
  - **File Types:** the input sheet. Yellow cells are required; header comments explain

each column.
    - **Lists:** hidden. Holds every dropdown's values and the template version.
- **Upload runs a dry run:** nothing is saved. Each row is checked for:
    - required values and valid list values
    - a client that belongs to the row's prod
    - a folder under the watched roots that exists
    - at least one file matching the pattern
    - a delimiter that splits the header
    - every named header present in the newest matching file (otherwise suggest the closest: *"Did you mean 'CO_DT'?"*)
    - a date format that reads ≥ 95% of sample dates (otherwise suggest the detected one)
    - an account column ≥ 99% filled
- **Row statuses: Ready**, **Will update** (replaces an existing setup for that folder and pattern), or **Needs fixing**.
- **Fix buttons** change one value and re-check that row on the server. Fixes are revealed in sequence; for example, once the header is fixed, a date-format problem may appear.
- **Save** commits only Ready and Will update rows (partial save is allowed), then reprocesses waiting files.
- **Download sheet with errors** returns the same workbook with Status and Message columns and red or green rows.
- Old template versions are rejected with *"This template is out of date. Download a new one and copy your rows into it."*

---

10. UI SPECIFICATION (source of truth for the frontend)

> Codex: build exactly this. When something is not specified, choose the simplest option that matches existing iResolve pages, and do **not** invent new patterns, colours or components.

10.1 Global rules

Layout

- Use the standard iResolve page shell: sidebar and top bar as existing modules do.
- Content max width 1440 px, centred. Page padding 24 px; 16 px under 1280 px wide.

- **Page header:** title (H1) on the left, at most one primary button on the right, and an optional one-line description under the title in muted text.
- Spacing scale 4 / 8 / 12 / 16 / 24 / 32 px. Cards 16 px padding, 8 px radius (or the iResolve card style if different).
- Minimum supported width 1280 px. At 1024–1279 px, tables scroll horizontally inside their card; the page never scrolls sideways.

One primary action per screen.

- Primary = filled button. Secondary = outlined. Tertiary = text link.
- Destructive actions (Retire, Cancel import) use a secondary button with a confirm dialog, never a red primary button.

Status chips: text plus colour, never colour alone. Same everywhere:

Status	Colour token	Label
processed	success	Processed
partial	warning	Done with errors
needs_review (file)	info	Waiting
needs_review (file type)	info	Needs setup
approved	success	Set up
failed	danger	Failed
headerless	neutral	Parked
retired	neutral	Retired
auto_drift version	warning	Layout changed (auto)

Confidence chips (suggestions only). Always show the reason in a tooltip and as muted text under the field on hover or focus:

conf	Label	Colour
HIGH	Looks right	success
MEDIUM	Please check	warning

LOW	Best guess	neutral
none	Not found	danger outline

Data display

- **Masking:** identifiers show only the last 4 characters (`******4821`); names show initials (`J*** D***`); dates are shown as-is. The browser never receives unmasked account values. All masking is done server-side.
- **Paths:** shortened to the last 3 parts with a leading `…\` (`…\Telecom\ATT\Wireless`). The full path is in a tooltip and copyable via a copy icon.
- **Numbers:** thousands separators (`318,220`). Percentages are whole numbers (`97%`).
- **Dates and times:** `22 Oct 2026, 10:30 IST` . Relative time ("5 min ago") is fine in the Overview only, with an absolute tooltip.

States: every data area must implement all four:

1. **Loading:** skeleton rows or blocks matching the final layout. No spinners over 300 ms on whole pages; a spinner only inside a button that is working.
2. **Empty:** an icon, one sentence saying why it's empty, and the next action as a button (copy is given per screen).
3. **Error:** an inline banner in the area, with the server message and a **Try again** link. Never a blank area.
4. **Data.**

Forms and validation

- Validate inline on change. Show field errors under the field in danger colour.
- Primary save buttons stay **disabled** until valid. Hovering the disabled button shows the first reason in a tooltip, and the same reason appears as muted text next to the button ("To save: Pick the Charge-off Date column.").
- Server errors (`{"errors": [...]}`) appear as a list in a danger banner above the footer buttons.

Export + log button pair (applies everywhere a run's export appears)

- Wherever a run's **Download extract (CSV/TXT)** button appears, a **Download log** button sits immediately to its right: same size, secondary style, icon 📄, label "Download log".
- **Extract disabled** when the run extracted 0 accounts, with the tooltip "No accounts in this run". **The log button stays enabled.**

- **Log missing** (deleted by retention): the log button is disabled with the tooltip "Log deleted after N days (retention setting)".
- **While a run is running:** the log button is enabled (it downloads the log so far) and a **View live log** link opens the Log viewer drawer (10.4).
- Downloads are plain links (`/runs/{id}/export.csv` , `/runs/{id}/log.txt`) so the browser handles the file.
- Places this applies:
  - Overview "Last processing run"
  - Run history rows
  - Files drawer (the run that processed the file)
  - Wizard step 2 (scan log only; scans have no extract)
  - Wizard step 4 (the reprocess run's extract and log)
  - Template import success toast ("View run")
  - Single-file reprocess result

Feedback

- Toasts bottom-right, 5 s, for completed actions ("Saved. 23 waiting files are being processed."). Errors are never toast-only: they also show inline.
- Long jobs show a progress card (10.4) that polls every 2 s via `useRunPolling` and keeps polling through network blips.
- Navigating away during a scan is allowed. The scan continues, and the Overview shows it under "Running now".

Accessibility

- Every control has a visible label.
- Chips have text.
- Focus rings are visible.
- Dialogs trap focus and close on Esc.
- Progress text uses `aria-live="polite"` .
- Colour contrast is at least 4.5:1.
- All actions are reachable by keyboard.

Keyboard shortcuts (setup screen only; show them on the button as a `kbd` hint):

- `A` saves and moves to the next file type.
- `1` – `9` assigns column N to the active field.
- `Esc` closes drawers and dialogs.

- Shortcuts are ignored while typing in an input.

Permissions in the UI

- Hide navigation items the user has no access to: Set Up needs `setup`; Settings needs `admin`.
- If a deep link is opened without access, show "You don't have access to this page. Ask an iResolve admin for Placement Intake setup access." The server still enforces everything.

10.2 Routes and navigation

```
Sidebar: Placement Intake
  Overview                        /modules/placement-intake
  Set Up File Types               /modules/placement-intake/setup
      Discover (wizard)           /modules/placement-intake/setup/discover[?run=I
      Excel Template              /modules/placement-intake/setup/template[?impor
  Needs Setup  (badge)            /modules/placement-intake/review
  Files                           /modules/placement-intake/files
  Run history                     /modules/placement-intake/runs
  File Types                      /modules/placement-intake/file-types[/ID]
  Settings  (admin only)          /modules/placement-intake/settings[?tab=...]
```

- **Badge** on Needs Setup = number of file types needing setup plus files waiting for a client, from `GET /review/`, refreshed on page focus. Hide the badge when it is 0.
- **Wizard URL state:** `run` and `step` live in the query string, so refresh, back and share all work. Browser Back moves one wizard step back.

10.3 Overview (`/modules/placement-intake`) — "Is everything OK?"

Data: `GET /overview/`. Refresh on page focus and every 60 s.

```
┌ Placement Intake ──────────────────────────────────────── [▶ Process new files
│ Extracts account, charge-off date and client from placement files.
├──────────────────┬──────────────────┬──────────────────┬──────────────────────
│ 1,284            │ 12               │ 3                │ 2
│ files processed  │ file types need  │ files failed     │ layouts changed
│ last 7 days      │ setup            │                  │ (handled automatically
│                  │ [Set them up →]  │ [View →]         │ [Review →]
├──────────────────┴──────────────────┴──────────────────┴──────────────────────
│ Running now: Scan #41 · 1,210 files read · 37 file types   [View]
├──────────────────────────────────────────────────────────────────────────────
│ Last processing run #41 · 22 Oct 2026, 10:30 IST · 42 files · 318,220 accounts
│ 1 failed · 1 done with errors
```

```
| [⬇️ Download extract (CSV)]  [📄 Download log]   View live log · All runs →
├──────────────────────────────────────────────────────────────────────────────
| Today by prod      Files   Accounts   Rows with errors
| attprod            20      190,002    412
| tmobile            18      120,400    0
└──────────────────────────────────────────────────────────────────────────────
```

- **Primary button:** `POST /runs/ {mode:"process"}`.
  - While a processing run is active, the button shows "Processing… 14 files" and is disabled.
  - A 409 response shows the inline message "A run is already in progress."
  - On finish, a toast reads "Processing finished: 42 files, 318,220 accounts".
- **Stat cards:** the whole card is clickable.
  - "need setup" → `/review`
  - "failed" → `/files?status=failed`
  - "layouts changed" → `/file-types?filter=changed`
- **Danger styling:** the failed card uses danger only when the count is > 0, and the needs-setup card uses info only when > 0. Otherwise both are neutral.
- **First run** (no runs and no file types), shown instead of the page body: "No placement files have been set up yet." with buttons [Discover file types] [Use Excel template].
- **No roots configured** (admins only): a warning banner "No placement folders are being watched. Add them in Settings."

10.4 Shared components

RunProgressCard (used by the wizard step 1 and Overview "Running now")

```
┌ Scanning folders… ──────────────────────────── 2 min ┐
| ████████████▒▒▒▒▒▒▒▒  1,210 files read · 37 file types|
| You can leave this page. The scan keeps running.     |
| View live log                                        |
└──────────────────────────────────────────────────────┘
```

- Indeterminate bar until the total is known (the server sends counts only), then a determinate bar.
- **Failed:** danger banner "The scan stopped: <error>" with [Try again] and [📄 Download log].
- **Done:** replaced by the next step's content. No extra click.

LogViewerDrawer (right drawer, 720 px; opened by "View log" / "View live log")

- Data: `GET /runs/{id}/log/?lines=500`. While the run is running, poll every 3 s and auto-scroll to the bottom, unless the user has scrolled up, in which case show a "Jump to latest" pill.
- Monospace, 12 px, no wrapping (horizontal scroll). Line colours by level: ERROR in danger, WARN in warning, INFO in default. Section lines (`---`) are bold with 8 px space above.
- **Toolbar:**
  - filter chips All / Warnings / Errors (client-side)
  - a search box that highlights matches
  - [📄 Download full log]
- **Note:** "Showing the last 500 lines" when truncated, with a link to download the full file.
- **Empty:** "This run has no log (it may have been deleted after the retention period)."

PathText: shortened path, full-path tooltip and copy icon.

StatusChip / ConfidenceChip: see 10.1.

EmptyState: icon, sentence and optional button. Centred in its card, 48 px vertical padding.

MaskedSampleTable

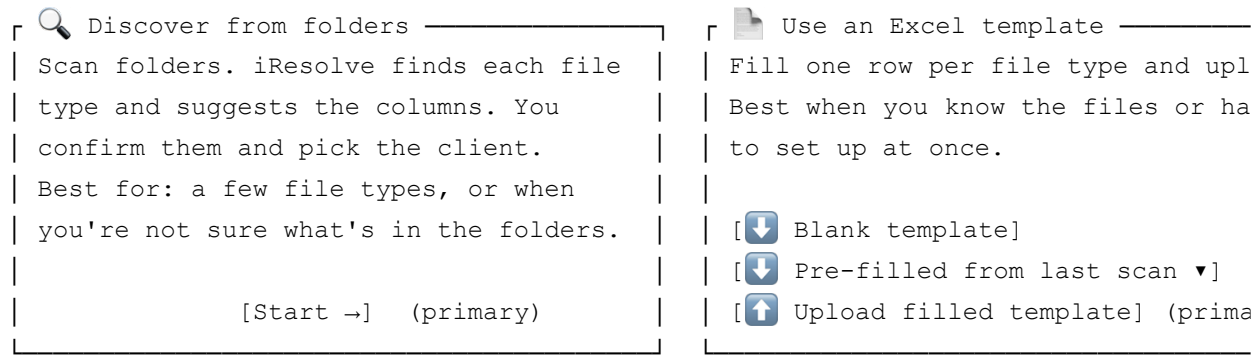
- Horizontally scrollable, with sticky first row (headers).
- Each header cell shows its column number (1–9 badge) and name, plus the assigned field name in a small chip if assigned.
- Assigned columns have a tinted background across all rows.
- Clicking a header cell assigns that column to the *active field*.
- Max 5 rows. Values in monospace.

ClientSelect: searchable combobox.

- Searches code and name, filtered by prod. Option format: **ATTW** ATT Wireless.
- Empty result: "No clients match. Clients are managed in Settings."
- Opens on click or Enter, with arrow-key navigation.

10.5 Set Up File Types landing (`/setup`)

Two equal cards, side by side (stacked under 1280 px). No other content.



- **"Pre-filled from last scan ▾"** is a menu of the last 5 completed scans, labelled "Scan #41 · 22 Oct · 37 file types". It is disabled with the tooltip "Run a scan first" if there are none.
- **Recent line:** links reopen that scan's step 2 or that import report.

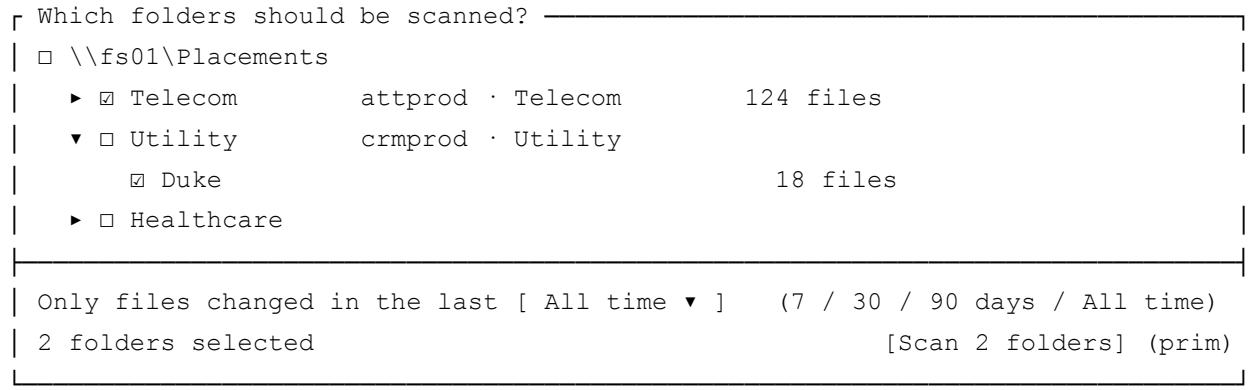
10.6 Discover wizard (`/setup/discover`)

Stepper: sticky at the top, with four steps:

`1 Choose folders → 2 Review scan → 3 Set up file types (n of N) → 4 Done .`

Completed steps are clickable; future steps are not.

Step 1 — Choose folders



- **Tree:** loaded lazily with `GET /folders/?path=` . Each node shows prod · segment (muted) when known and its file count.
- **Checkbox behaviour:** checking a parent checks its children; the parent shows a tri-state when partly checked. Only the top-most checked folders are sent as `roots` .
- **Starting the scan:** `POST /runs/ {mode:"discovery", roots, since_days}` . Then show the RunProgressCard in place of the tree and put `?run=ID` in the URL.

- **Empty roots:** "No placement folders are being watched yet." plus [Go to Settings] for admins, or "Ask an admin to add placement folders" for everyone else.

Step 2 — Review scan

Data: `GET /runs/{id}/formats/` .

```
Scan #41 · 1,284 files · 37 file types · 5 parked · 3 skipped · 1 folder not acce
┌ Needs setup (12) ──────────────────────────────────────────────────────────────
│ ● ATT Wireless – ATT_PLC_*   23 files · 2 folders · 3 of 3 columns suggested
│   Confidence: Looks right                                          [Set up →]
│ ● Duke – DUKE_NEW            9 files · 1 folder · 2 of 3 columns suggested
│   Confidence: Please check                                         [Set up →]
├ Layout changed (2) ────────────────────────────────────────────────────────────
│ ● ATT Wireless – ATT_PLC_* (changed layout)  4 files               [Set up →]
├ Already set up (23)  ▸ (collapsed) ────────────────────────────────────────────
├ Parked — not supported yet (5)  ▸ (collapsed) ──────────────────────────────
│   …\Health\HCA\HCA_0921.dat   No header row (fixed width) — Phase 2
└─────────────────────────────────────────────────────────────────────────────

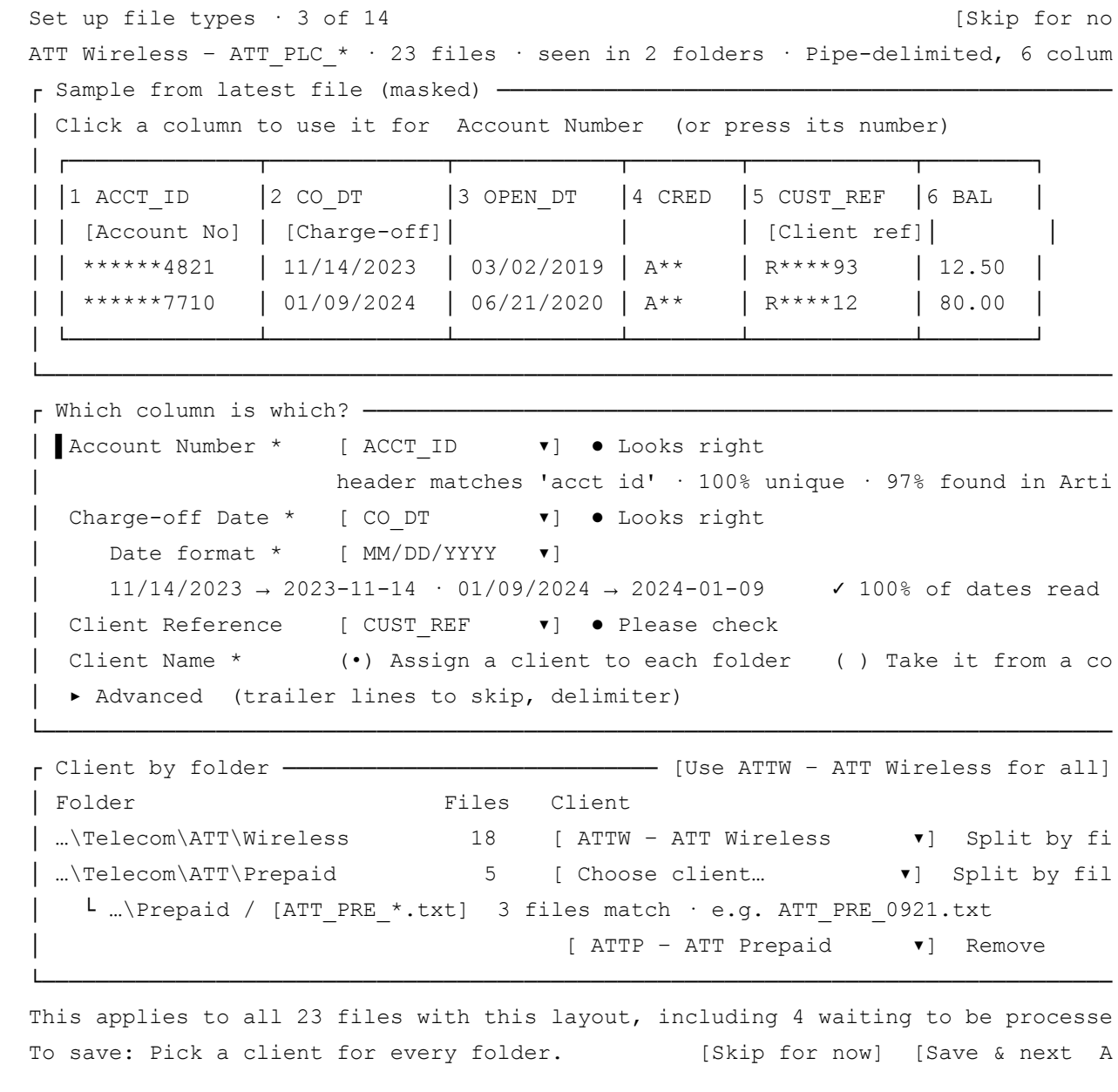
[Download as Excel template]            [Set up all 14 one by one →] (primary)
```

- **Order:** Needs setup (most files first), then Layout changed, then Already set up (collapsed), then Parked or skipped (collapsed). Parked rows show the reason in plain words:
  - `headerless` → "No header row"
  - `no_delimiter_possible_fixed_width` → "Fixed width"
  - `skipped_ext.xlsx` → "Excel file — not supported yet"
  - `empty` → "Empty file"
  - `error:*` → "Could not be read"
- **Buttons:**
  - **Primary** starts step 3 at the first Needs setup item and walks through the Needs setup and Layout changed items in order.
  - **"Download as Excel template"** = `GET /template/?run_id=ID` , for users who would rather fill it in Excel.
- **Nothing to set up:** "Every file type in these folders is already set up." with [Process new files now] and [Back to Overview].

Step 3 — Set up a file type (`FileTypeSetup.jsx`)

Data: `GET /formats/{id}/` (detail) and `GET /formats/{id}/sample/` (masked rows), loaded

in parallel, with skeletons while loading.



Behaviour (all required):

- **Active field**
  - One field row is active at a time: Account Number on load, shown with a left accent bar.
  - Clicking or focusing a field row makes it active.
  - Clicking a sample column header, or pressing `1` – `9`, assigns that column to the active field and then moves the active field to the next empty required field.
- **One column per field**
  - Assigning a column already used by another field moves it: the other field becomes

empty. Never allow duplicates.
    - In dropdowns, used columns show "(used)" but stay selectable, and selecting one moves it.
- **Dropdowns**
    - Required fields start with "Choose a column…".
    - Client Reference's empty option is "Not in file".
- **Suggestion chip and reason**
    - Shown only while the value equals the suggestion and the user hasn't changed that field.
    - Once changed, the chip disappears; no "Custom" chip.
- **Date preview**
    - Calls `GET /formats/{id}/date-preview/?header=&date_format=` 300 ms after either value changes.
    - Shows up to 3 `raw → parsed` pairs and "N% of dates read".
    - **Below 95%:** red text, Save is blocked, and if the server suggests another format, a link "Use YYYY-MM-DD instead".
    - Blank or unreadable values show *can't read* in italics.
- **Client Name**
    - **Assign a client** (default) shows the Client by folder table.
    - **Take it from a column** hides the table and shows a column dropdown.
- **Client by folder**
    - One row per folder where this type was seen, with its file count.
    - "Use <first chosen client> for all" appears when there are 2 or more folders and at least one is chosen.
    - **"Split by file name"** adds an indented sub-row with a pattern input (placeholder `e.g. ATT_PRE_*.txt`).
        - 400 ms after typing, it calls `POST /formats/{id}/pattern-test/` and shows "N files match · e.g. NAME". 0 matches shows warning text "No files match", which does not block saving.
        - Each sub-row has its own client and a Remove link.
- **Advanced** (collapsed): trailer lines to skip (0–10), plus a read-only delimiter and column count.
- **Save & next**

- Disabled until valid (rules in `FileTypeSetup.jsx::problems`); the first problem shows next to the button.
- On success: toast "Saved ATT Wireless – ATT_PLC_*. 4 waiting files are being processed.", then load the next item without a full page reload, and update the stepper "n of N".
- After the last item, go to step 4.

- **Skip for now:** moves to the next item without saving; the item stays in Needs Setup.
- **Leaving with changes:** if the user changed anything and navigates away, show the browser confirm "You have unsaved changes on this file type."
- **Sample unavailable:** if the latest file can't be read, show the info banner "The latest file for this type can't be opened right now. You can still pick columns by name." The dropdowns still work from `detail.headers`.

Step 4 — Done

```
✓ 12 file types set up · 2 skipped
47 waiting files are being processed now.
Processing run #42 · running…   [⬇️ Download extract (CSV)] [📄 Download log] · \
[Go to Files]   [Set up skipped (2)]   [Back to Overview] (primary)
```

10.7 Excel Template page (`/setup/template`)

State A — Start

```
┌ Set up file types with Excel ──────────────────────────────────────────────
│ 1. Download a template
│    [⬇️ Blank template]   [⬇️ Pre-filled from scan ▾]
│    Pre-filled is faster: folders, file names and suggested columns are already
│ 2. Fill one row per file type per folder. Yellow columns are required.
│ 3. Upload it. iResolve checks every row against a real file before saving.
│    ┌ - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
│    │      Drop the filled .xlsx here, or [choose file]
│    └ - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
└────────────────────────────────────────────────────────────────────────────
```

- **Drop zone:** accepts a single `.xlsx`. Anything else shows "Upload the .xlsx template file."
- **While uploading:** "Checking 42 rows against the files…" with an indeterminate bar. The check is synchronous and can take ~1 s per row; show elapsed seconds.
- **Upload error** (wrong template, old version, >5 MB): danger banner with the server

message, and the drop zone stays.

State B — Import report (`TemplateImportReport.jsx` , URL `?import=ID` so a refresh keeps it):

```
Checked: att_types.xlsx
[All 48] [Ready 38] [Will update 4] [Needs fixing 6]     ← tabs; opens on "Needs
```

Row	Folder / file name	Client	Files	Status	What to
7	…\Telecom\ATT ATT_PLC_*.txt	ATTW	12	Needs fixing	Charge- not fou [Use CO
12	…\Utility\Duke *.csv	DUKE	9	Needs fixing	Date fo 0% of t YYYY-MM
15	…\Health\HCA HCA_*.txt	HCA1	—	Needs fixing	No file 'HCA_*.

```
(sticky footer)  [Download sheet with errors marked]   [Cancel]   [Save 42 file t
```

- **Fix button** → `POST /template/imports/{id}/fix/` . The button shows "Checking…" and the row updates in place; counts and tabs update. If the row becomes Ready, it stays visible in the current tab until the tab is changed, so it doesn't jump away.
- **Errors without a fix button** (folder not found, no matching files) say what to change in Excel. The user fixes them in the downloaded errors sheet and uploads again.
- **Save** opens a confirm dialog: "Save 42 file types? 38 new, 4 replacing an existing setup. 6 rows with problems will not be saved. Files already waiting in these folders will be processed straight after." [Back] [Save].
  - On success: toast "42 file types saved. Waiting files are being processed.", then go to Files filtered to Waiting/Processing.
- **Cancel** marks the import cancelled and returns to State A.

10.8 Needs Setup (`/review`)

The same list style as wizard step 2, but across all scans:

```
Needs setup (12)                                          Sort: [Most files ▾ | Ne
● ATT Wireless – ATT_PLC_*   23 files · 4 waiting · …\Telecom\ATT (2 folders)   [
● Duke – DUKE_NEW             9 files · 9 waiting · …\Utility\Duke              [
Files waiting for a client (3)
● …\Telecom\ATT\Prepaid\ATT_PRE_1001.txt — No client assigned for this file name
```

- **[Set up →]** opens the same `FileTypeSetup` screen as a full page (`/file-types/{id}/setup`). After saving, it returns to this list, and the item disappears with a 200 ms fade.
- **[Assign →]** for files missing a client opens that file type's setup scrolled to Client by folder.
- **Empty:** "Nothing needs setup. New file types will appear here automatically." [Back to Overview].

10.9 Files (`/files`)

```
Files                                                    [⬇️ Export filtered (
[Search file name…]  Status [All ▾]  Prod [All ▾]  Date [Last 7 days ▾]   Clear f
┌──────────────────────┬──────────────────┬──────────┬────────────┬──────────────
│ File                 │ Folder           │ Prod     │ Client     │ Status
│ ATT_PLC_20261021.txt │ …\Telecom\ATT    │ attprod  │ ATT Wire   │ Done w/ errs
│ ATT_PRE_1001.txt     │ …\ATT\Prepaid    │ attprod  │ —          │ Waiting
└──────────────────────┴──────────────────┴──────────┴────────────┴──────────────
                                                  ‹ 1 2 3 … 26 ›   50
```

- **Data:** `GET /files/?status=&prod=&q=&from=&to=&page=` . Filters live in the URL query string. Search is debounced by 300 ms.
- **Status filter** is multi-select using the UI labels from 10.1.
- **Row click** opens a right-side **drawer** (480 px):
  - header: file name, full path with copy, and status chip
  - message (if any)
  - counts
  - client
  - file type link
  - version used
  - **Row errors** section (`GET /files/{id}/errors/`): a summary by error message with counts, then a table of **Account # (serial in file)**, line number, field and message (first 1,000, with the note "Showing first 1,000 of N")
- **Drawer actions:**
  - [⬇️ Download extract] (`/files/{id}/export.csv`) and [📄 Download log] for **the run that processed this file** (`/runs/{run_id}/log.txt`), plus "View log" (opens the Log viewer drawer)
  - [Reprocess] (admin only): confirm, then show the result inline with that reprocess run's own [📄 Download log]

- [Set up file type] when the file is Waiting

- **Empty states:**
  - no files at all: "No files processed yet."
  - filters match nothing: "No files match these filters." [Clear filters]

10.9b Run history (`/runs`)

```
Run history                                          Type [All ▾ | Processing | Sca
```

Run	Type	Started	Status	Result
#42	Reprocess	22 Oct 10:41	Running	12 files so far
#41	Processing	22 Oct 10:30 by schedule	Done	42 files · 318,220 accounts · 1 failed · 1 with errors
#40	Scan	22 Oct 09:58	Done	1,284 files · 37 file types
#39	Processing	22 Oct 10:00	Failed	Share not reachable

- **Data:** `GET /runs/list/?mode=` , the last 100 runs, newest first.
- **Columns:**
  - Started shows "by <user>" or "by schedule" on a second line.
  - Status uses chips: Running (info, pulsing dot), Done (success), Failed (danger).
  - Result is a one-line summary built from `stats` ; Failed shows `error` .
- **Buttons:** follow the export + log pair rule. Scans have no Extract button.
- **Empty:** "No runs yet." [Process new files now].
- Linked from Overview ("All runs →") and from the sidebar.

10.10 File Types library (`/file-types` , `/file-types/{id}`)

- **List:**
  - Columns: name, status chip, files, folders (first one plus "+N"), last seen, version.
  - Search across name, folder and header text. Filter by status and "Layout changed".
  - Row click opens the detail page.
- **Detail page:**
  - **Header:** name, status and file count. Actions: [Edit setup] (opens FileTypeSetup pre-filled with the current version), [Retire] (admin, with the confirm "Retire this file type? New files with this layout will wait for setup again.").
  - **"Columns" card:** read-only table of field → column (with "Client: assigned per

folder" or the column name) and the date format.

- **"Client by folder" card:** read-only table of folder, pattern and client.
- **"Versions" card:** list of v3 · Edited by kyash · 22 Oct · Wizard (or Excel template, Automatic, Layout changed (auto)). Expanding a version shows what changed from the previous one in plain words ("Charge-off Date: OPEN_DT → CO_DT").
- **"Headers in this file type" card:** a chip list of all headers.
- **Advanced (collapsed):** signature and delimiter.

10.11 Settings (`/settings` , admin only)

Tabs: **Folders & schedule · Clients · Segments · Header names · Export**.

Tab	Content
Folders & schedule	Editable list of watched root folders (UNC). Adding one validates that it exists ("iResolve can't open this folder"). Processing schedule: every [30 ▾] minutes / Off. **Auto-approve** toggle with the help text "Set up new file types automatically when iResolve is sure about the columns and the folder already has a client. Off is safer."
Clients	Table of code, name, prod and active, with search. [⬆️ Import CSV] (columns `client_code, client_name, prod`) shows the result "Saved 120 of 120 rows".
Segments	Table of folder, prod, segment and date format. [⬆️ Import CSV] (`folder_path, prod, segment, date_format`).
Header names	Grouped by field. Chips show the alias plus `seed` or `learned` (with who and when). Learned aliases have an × to remove, with confirm.
Export & logs	Export folder (UNC, validated) and format radio: CSV (comma) / TXT (pipe). Preview line of the output columns: `file_name, file_path, account_seq, line_number, account_number, chargeoff_date, client_name, client_ref` . **Log folder** (UNC; blank = default `<app>\logs\placement_intake` , shown as placeholder). **Keep logs for** [90] days (0 = forever), with the help text "Older logs are deleted automatically after each processing run."

Every tab saves with its own **Save** button (disabled until changed), followed by the toast "Settings saved".

10.12 UI acceptance checklist (Codex: verify every item)

- □ No screen shows signature, JSON, regex or internal status names. UI words follow section 3.
- □ Every list or table has loading, empty (with the given copy), error (with Try again) and data states.
- □ Unmasked account numbers never reach the browser (check the network tab on the setup screen).
- □ Clicking a sample column assigns it to the active field; `1` – `9` work; a column can't be assigned to two fields.
- □ The date preview updates within about 0.5 s of a change; < 95% blocks Save and offers the suggested format.
- □ Save buttons are disabled with a visible reason until valid.
- □ The split-by-file-name row shows the live match count.
- □ A wizard refresh at any step restores the same step (URL state).
- □ Template import: fix buttons update the row in place; Save commits only valid rows; the errors sheet downloads.
- □ Badges and counts match between Overview, Needs Setup and Files.
- □ Everything works with the keyboard alone; focus is visible; dialogs trap focus and close on Esc.
- □ At 1280 px and 1024 px nothing overlaps; tables scroll inside their card.
- □ A user without `setup` permission can't see or reach Set Up pages, and the API returns 403.
- □ Every run export button has a Download log button right next to it; the log button works while a run is running and after a failed run.
- □ The Log viewer polls while running, colours WARN/ERROR lines, filters and searches, and never shows account values.
- □ Export files include `account_seq` and `line_number`; the Files error table shows the Account # (serial) of each rejected row.

---

11. Phase 2 hook — fixed-width and multi-record files

- Already detected and stored as **Parked** (`headerless` , `no_delimiter_possible_fixed_width`).
- **Signature:** a hash of record-type codes plus the line length per record type, e.g. `H:120|D:450|T:80` .
- **Config:** `parse_config` gains `record_type_slice` and per-record lengths. Field maps

use `{"source":"position","record_type":"D","start":12,"length":20}`.

- **UI:** a ruler view of a masked line where the user drags to select a span, then the same Client by folder table.

12. API

All endpoints are under `/api/placement-intake/` (`urls.py`). Errors return `{"errors":["..."]}` with status 400, 403, 404 or 409.

Method & path	Perm	Purpose
GET `overview/`	view	Overview numbers
POST `runs/ {mode, roots?, since_days?}`	setup	Start a scan or processing run → 202 `{run_id}`; 409 if one is already running
GET `runs/{id}/`	view	Status, progress, stats, error
GET `runs/{id}/formats/`	view	Wizard step 2 list plus parked files
GET `runs/{id}/export.csv`	view	Extract for that run
GET `runs/{id}/log.txt`	view	Download the run's log file (works while running)
GET `runs/{id}/log/?lines=500`	view	Last N log lines + run status (Log viewer)
GET `runs/list/?mode=`	view	Run history (last 100)
GET `folders/?path=`	setup	Folder tree (roots when no path)
GET `formats/?status=&q=&sort=`	view	File types list
GET `formats/{id}/`	view	Detail: headers, suggestion, current mapping, clients, versions
GET `formats/{id}/sample/`	setup	Masked sample rows
GET `formats/{id}/date-preview/?header=&date_format=`	setup	Live date preview
POST `formats/{id}/pattern-test/`	setup	File-name pattern match count

POST `formats/{id}/setup/`	setup	Save wizard setup (payload in `services.save_format_setup`)
POST `formats/{id}/retire/`	admin	Retire a file type
GET `review/`	view	Needs Setup list plus files missing a client
GET `files/?status=&prod=&q=&from=&to=&page=`	view	Files table
GET `files/{id}/errors/`	view	Row errors
POST `files/{id}/reprocess/`	admin	Reprocess one file
GET `files/{id}/export.csv`	view	Extract for one file
GET `clients/?prod=&q=`	view	Client combobox
GET/PUT `settings/`	admin	Settings
GET `settings/segments/`, GET/DELETE `settings/synonyms/`	admin	Reference data
POST `settings/import/{clients|segments}/`	admin	CSV import
GET `template/?run_id=`	setup	Blank or pre-filled template (.xlsx)
POST `template/imports/` (multipart `file`)	setup	Upload and dry run → report
GET `template/imports/{id}/`	setup	Report
POST `template/imports/{id}/fix/` `{row_id, field, value}`	setup	Apply a fix and re-check the row
POST `template/imports/{id}/commit/`	setup	Save valid rows
POST `template/imports/{id}/cancel/`	setup	Cancel
GET `template/imports/{id}/errors.xlsx`	setup	Workbook with errors marked

13. Security / HIPAA

- **Masking:** samples are masked server-side and never stored. The only raw values sent to the browser are dates from the chosen date column (date preview).
- **What is stored:** discovery results, templates, row errors and **run log files** hold headers, statistics, account serial and line numbers, and codes only, never account values. The log folder should be on a restricted share like the export folder.
- **Artiva:** read-only, sampled, rates-only logging.
- **Access:**
  - Exports go only to the configured folder.
  - Folder browsing and pattern tests are restricted to the watched roots, which blocks path traversal.
  - Every endpoint checks permission server-side.
- **Audit:** every mapping version records who approved it and how.

14. Build order for Codex

1. Run the dry run (no Django): `python -m placement_intake.cli_discover --root "\\fs01\Placements" --out .\discovery`, and review `formats.csv`.
2. Copy the 🟢 files unchanged. Add unit tests (section 15).
3. Adapt 🟡 `models.py` and create migrations. Run `python manage.py pi_seed`.
4. Wire `permissions.user_has_perm`, the Celery app and beat schedule, and the Artiva connection plus `ARTIVA_LOOKUP`.
5. Adapt `services.py`, `tasks.py`, `views.py` and `urls.py`. Test every endpoint with the fixtures.
6. **UI order:** Settings (roots, clients, segments) → Discover wizard (steps 1–4; build `FileTypeSetup` with care) → Excel template page → Needs Setup → Files → File Types → Overview.
7. Go through the acceptance checklist in 10.12 and the backend acceptance below.

Celery beat (adapt to iResolve settings):

```
CELERY_BEAT_SCHEDULE = {
    "placement-intake-process": {"task": "placement_intake.process_scheduled", "s
}
```

Backend acceptance:

- Every historical flat file ends in a visible state with a reason.
- Re-running processing creates no duplicates (hash check), and every skipped file is listed in the log.
- The run's export row count equals the `pi_accounts` count for that run.
- Saving a file type processes its waiting files.
- A missing mapped header fails the file with a clear message, never shifting columns.

15. Tests Codex must add (pytest)

For the 🟢 modules, using synthetic files:

- pipe, comma, tab and tilde delimiters
- BOM
- quoted fields with embedded delimiters
- trailer row
- blank account, duplicate account, bad date, future date
- two date columns, where the earliest must win
- numeric IDs that look like `%Y%m%d`
- headerless and fixed-width files
- a missing mapped header must raise `MappingError`
- template round trip: build → fill → read → validate, including a near-miss header, a wrong date format, a non-existent folder, a pattern with no matches, and a client from another prod
- `run_log` : a log is written for a normal run and for a crashing run (footer + traceback), counts are correct, `prune_logs` respects retention, and no account values appear in any log (grep the log for the fixture's account numbers; expect zero hits).
- `account_seq` : it equals the record position, excluding trailers; rejected rows keep their serial; the export has gaps where rows were rejected.

Services: an idempotent rerun, drift auto-reuse, the client lookup order (pattern beats folder), waiting files processed after setup, and the log path being stored on the run at start.

16. Open items to confirm

- □ DRF or plain Django views (section 0)
- □ Artiva table and columns for client account number and charge-off date, per prod
- □ Client list source: Artiva client table (which) or a maintained CSV

- ☐ `openpyxl` in the bundle
- ☐ Any `.xlsx`, `.zip` or PGP placement files? These are currently skipped and shown as such.
- ☐ Should processed files be archived or moved? Currently they are left in place.
- ☐ Export folder; per prod or combined?
- ☐ Is `client_ref` required for any client?

Appendix A — Dry run output example

```
> python -m placement_intake.cli_discover --root \\fs01\Placements --out .\discov
1284 files, 37 layouts
files scanned: 1284   distinct layouts: 37
  ok: 1271
  headerless: 5
  skipped_ext.xlsx: 3
  ...
```

`formats.csv` columns: signature, file_count, folders, delimiter, column_count, headers, latest_example, first_seen, last_seen, overall_confidence, date_format, then `<field>_header`, `<field>_conf` and `<field>_reason` for each field.

Appendix B — Code

Files are listed in build order. `__init__.py` files (app package and `management/`, `management/commands/`) are empty and not shown.

Contents

1. `placement_intake/common.py` — 🟢 DROP-IN · TESTED
2. `placement_intake/discovery.py` — 🟢 DROP-IN · TESTED
3. `placement_intake/automap.py` — 🟢 DROP-IN · TESTED
4. `placement_intake/parser.py` — 🟢 DROP-IN · TESTED
5. `placement_intake/template_io.py` — 🟢 DROP-IN · TESTED
6. `placement_intake/run_log.py` — 🟢 DROP-IN · TESTED
7. `placement_intake/cli_discover.py` — 🟢 DROP-IN · TESTED
8. `placement_intake/artiva_check.py` — 🟡 ADAPT · NOT RUN

9. `placement_intake/apps.py` — 🟡 ADAPT · NOT RUN
10. `placement_intake/models.py` — 🟡 ADAPT · NOT RUN
11. `placement_intake/services.py` — 🟡 ADAPT · NOT RUN
12. `placement_intake/tasks.py` — 🟡 ADAPT · NOT RUN
13. `placement_intake/permissions.py` — 🟡 ADAPT · NOT RUN
14. `placement_intake/views.py` — 🟡 ADAPT · NOT RUN
15. `placement_intake/urls.py` — 🟡 ADAPT · NOT RUN
16. `placement_intake/management/commands/pi_seed.py` — 🟡 ADAPT · NOT RUN
17. `frontend/placement-intake/api.js` — 🟠 REFERENCE UI · NOT COMPILED
18. `frontend/placement-intake/useRunPolling.js` — 🟠 REFERENCE UI · NOT COMPILED
19. `frontend/placement-intake/FileTypeSetup.jsx` — 🟠 REFERENCE UI · NOT COMPILED
20. `frontend/placement-intake/TemplateImportReport.jsx` — 🟠 REFERENCE UI · NOT COMPILED

B.1 `placement_intake/common.py`

> 🟢 **DROP-IN · TESTED**

```
"""Shared helpers: normalization, signatures, dates, masking, hashing. Stdlib onl
import fnmatch
import hashlib
import re
from datetime import date, datetime

FIELDS = ["account_number", "chargeoff_date", "client_name", "client_ref"]
REQUIRED_FIELDS = ["account_number", "chargeoff_date"]
FIELD_LABELS = {
    "account_number": "Account Number",
    "chargeoff_date": "Charge-off Date",
    "client_name": "Client Name",
    "client_ref": "Client Reference",
}

# Friendly label (shown in UI / Excel) -> strptime code
DATE_FORMATS = {
    "MM/DD/YYYY": "%m/%d/%Y",
    "YYYY-MM-DD": "%Y-%m-%d",
    "MMDDYYYY": "%m%d%Y",
    "YYYYMMDD": "%Y%m%d",
    "MM/DD/YY": "%m/%d/%y",
```

```
    "MM-DD-YYYY": "%m-%d-%Y",
    "DD-MON-YYYY": "%d-%b-%Y",
    "DD-MON-YY": "%d-%b-%y",
    "YYYY/MM/DD": "%Y/%m/%d",
    "MM/DD/YYYY HH:MM:SS": "%m/%d/%Y %H:%M:%S",
    "YYYY-MM-DD HH:MM:SS": "%Y-%m-%d %H:%M:%S",
}
DATE_CODES = list(DATE_FORMATS.values())
DATE_LABEL_BY_CODE = {v: k for k, v in DATE_FORMATS.items()}

# Friendly label -> delimiter character
DELIMITERS = {"Comma": ",", "Pipe": "|", "Tab": "\t", "Semicolon": ";", "Tilde":
DELIMITER_LABEL = {v: k for k, v in DELIMITERS.items()}

FLAT_EXTS = {".txt", ".csv", ".dat", ".psv", ".tsv"}
SKIP_EXTS = {".xlsx", ".xls", ".zip", ".pgp", ".gpg", ".pdf"}


def norm(s: str) -> str:
    """Normalize a header for comparison: lowercase, non-alphanumerics -> space."
    return re.sub(r"\s+", " ", re.sub(r"[^a-z0-9]+", " ", (s or "").lower())).str


def signature(headers) -> str:
    """Identity of a flat-file layout: same headers in the same order -> same sig
    return hashlib.sha1("|".join(norm(h) for h in headers).encode()).hexdigest()[


def parse_date(value: str, codes=None):
    v = (value or "").strip()
    if not v:
        return None
    for code in codes or DATE_CODES:
        try:
            d = datetime.strptime(v, code).date()
        except ValueError:
            continue
        # Guard: pure-digit IDs like 7000101 parse as %Y%m%d (year 7000).
        if 1900 <= d.year <= 2100:
            return d
    return None


def detect_date_format(values):
    """Return (best_code, parse_rate) for a list of strings. Ties prefer list ord
    filled = [v.strip() for v in values if v and v.strip()]
```

```
    if not filled:
        return None, 0.0
    best_code, best_rate = None, 0.0
    for code in DATE_CODES:
        ok = sum(1 for v in filled if parse_date(v, [code]))
        rate = ok / len(filled)
        if rate > best_rate:
            best_code, best_rate = code, rate
    return best_code, best_rate


def wildcard_to_regex(pattern: str) -> str:
    """'ATT_PLC_*.txt' -> anchored, case-insensitive regex string."""
    return "(?i)" + fnmatch.translate(pattern.strip())


def sha256_file(path: str, chunk: int = 1 << 20) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as fh:
        for block in iter(lambda: fh.read(chunk), b""):
            h.update(block)
    return h.hexdigest()


def mask(value: str) -> str:
    """Mask a sample value for display. Dates pass through; IDs keep last 4; text
    v = (value or "").strip()
    if not v:
        return ""
    if parse_date(v):
        return v
    if re.fullmatch(r"[A-Za-z0-9\-]+", v) and any(c.isdigit() for c in v):
        keep = 4 if len(v) > 6 else 2
        return "*" * (len(v) - keep) + v[-keep:]
    if re.fullmatch(r"[\d.,$\-]+", v):
        return v  # amounts are not identifiers
    return " ".join(w[0] + "*" * (len(w) - 1) for w in v.split())


def today() -> date:
    return date.today()
```

B.2 `placement_intake/discovery.py`

> 🟢 **DROP-IN · TESTED**

```
"""Scan folders, read a sample of each flat file, and group files by header signa

Pure functions (no DB) so they can be used by the Celery task, the template impor
and the standalone CLI.
"""
import csv
import os
import re
from collections import Counter, defaultdict
from datetime import datetime, timedelta

from .common import DELIMITERS, FLAT_EXTS, SKIP_EXTS, parse_date, signature

SNIFF_DELIMS = list(DELIMITERS.values())


def sniff_delimiter(lines):
    """Pick the delimiter that appears the same (non-zero) number of times on the
    best, best_n = None, 0
    lines = [ln for ln in lines if ln.strip()]
    for d in SNIFF_DELIMS:
        counts = [ln.count(d) for ln in lines]
        if counts and min(counts) > 0 and len(set(counts[:5])) == 1 and counts[0]
            best, best_n = d, counts[0]
    return best


def looks_like_header(cells):
    """Headers are mostly text: few cells parse as dates or numbers."""
    if not cells:
        return False
    datey = sum(1 for c in cells if parse_date(c))
    numy = sum(1 for c in cells if re.fullmatch(r"[\d.,$\-]+", c.strip() or "x"))
    return (datey + numy) / len(cells) < 0.3


def read_sample(path, sample_rows=500, delimiter=None, header_row=0, encoding="ut
    """Return (info, status).

    info = {delimiter, headers, columns (list of value lists), rows}
    status = ok | empty | headerless | no_delimiter_possible_fixed_width
    """
    with open(path, "r", encoding=encoding, errors="replace", newline="") as fh:
        head = [fh.readline() for _ in range(header_row + 6)]
        delim = delimiter or sniff_delimiter(head[header_row:])
        if not delim:
```

```
            return None, "no_delimiter_possible_fixed_width"
        fh.seek(0)
        reader = csv.reader(fh, delimiter=delim)
        rows = []
        for i, row in enumerate(reader):
            if i < header_row:
                continue
            rows.append(row)
            if len(rows) > sample_rows:
                break
    if not rows or not any(c.strip() for c in rows[0]):
        return None, "empty"
    if not looks_like_header(rows[0]):
        return None, "headerless"
    headers = [h.strip() for h in rows[0]]
    body = [r for r in rows[1:] if len(r) == len(headers)]
    columns = [[r[i] for r in body] for i in range(len(headers))]
    return {"delimiter": delim, "headers": headers, "columns": columns, "rows": l


def iter_files(roots, since_days=None, on_error=None):
    """Yield (root, path) for every file under the roots.

    on_error(path, message) is called for folders that cannot be opened (missing
    no permission) so they can be logged instead of silently skipped.
    """
    cutoff = None
    if since_days:
        cutoff = (datetime.now() - timedelta(days=since_days)).timestamp()
    for root in roots:
        if not os.path.isdir(root):
            if on_error:
                on_error(root, "folder not found or not accessible")
            continue
        walk_err = (lambda e: on_error(getattr(e, "filename", root), str(e))) if
        for dirpath, _, files in os.walk(root, onerror=walk_err):
            for fn in files:
                path = os.path.join(dirpath, fn)
                if cutoff and os.path.getmtime(path) < cutoff:
                    continue
                yield root, path


def classify_file(root, path, sample_rows=500):
    """Read one file and return a record describing it (no raw values kept beyond
    fn = os.path.basename(path)
```

```
    ext = os.path.splitext(fn)[1].lower()
    rec = {
        "path": path,
        "file_name": fn,
        "folder": os.path.dirname(path),
        "root": root,
        "mtime": datetime.fromtimestamp(os.path.getmtime(path)),
        "size": os.path.getsize(path),
        "signature": None,
        "status": "ok",
        "info": None,
    }
    if ext in SKIP_EXTS or ext not in FLAT_EXTS:
        rec["status"] = f"skipped_ext{ext}"
        return rec
    try:
        info, status = read_sample(path, sample_rows)
    except Exception as exc:  # noqa: BLE001 - one bad file must not stop a scan
        info, status = None, f"error:{type(exc).__name__}"
    rec["status"] = status
    if info:
        rec["info"] = info
        rec["signature"] = signature(info["headers"])
    return rec


def scan(roots, since_days=None, sample_rows=500, progress=None, on_error=None):
    """Scan roots and group by signature.

    Returns {
      "files": [rec without info],
      "formats": {signature: {headers, delimiter, file_count, folders: Counter,
                              latest: rec (with info), first_seen, last_seen}}
    }
    """
    files, formats = [], {}
    by_sig = defaultdict(list)
    for n, (root, path) in enumerate(iter_files(roots, since_days, on_error), sta
        rec = classify_file(root, path, sample_rows)
        if rec["signature"]:
            by_sig[rec["signature"]].append(rec)
        files.append({k: v for k, v in rec.items() if k != "info"})
        if progress and n % 25 == 0:
            progress(n, len(by_sig))

    for sig, recs in by_sig.items():
```

```
        recs.sort(key=lambda r: r["mtime"])
        latest = recs[-1]
        # Keep only the latest file's sample in memory.
        for r in recs[:-1]:
            r["info"] = None
        formats[sig] = {
            "signature": sig,
            "headers": latest["info"]["headers"],
            "delimiter": latest["info"]["delimiter"],
            "file_count": len(recs),
            "folders": Counter(r["folder"] for r in recs),
            "latest": latest,
            "first_seen": recs[0]["mtime"],
            "last_seen": latest["mtime"],
        }
    if progress:
        progress(len(files), len(formats))
    return {"files": files, "formats": formats}


def status_counts(files):
    return Counter(f["status"] for f in files)
```

B.3 `placement_intake/automap.py`

> 🟢 **DROP-IN · TESTED**

```
"""Suggest which column is which canonical field, with a confidence and plain-lan

Signals: header synonyms, value profile, and (optionally) Artiva match rates.
The result only PRE-FILLS the UI / template; a human confirms unless auto-approve
"""
import difflib
import re
import statistics
from collections import Counter

from .common import FIELDS, detect_date_format, norm, parse_date, today

SEED_SYNONYMS = {
    "account_number": ["account number", "acct no", "acct num", "account no", "ac
                       "account", "accountnumber", "client account", "original ac
                       "acct id", "account id", "acct nbr", "accountnum", "acctno
                       "acct number", "account num"],
    "chargeoff_date": ["charge off date", "chargeoff date", "co date", "c o date"
```

```
                      "chg off date", "chargeoff dt", "co dt", "charged off date
                      "chargeoffdate", "charge off dt", "chgoffdt", "chg off dt"
    "client_name":    ["client name", "client", "creditor", "creditor name",
                      "original creditor"],
    "client_ref":     ["client reference", "client ref", "client ref no",
                      "reference number", "ref no", "client reference number", "
                      "reference id", "refid", "cust ref", "customer ref",
                      "customer reference", "client ref id"],
}


def profile_column(values):
    vals = [v.strip() for v in values]
    filled = [v for v in vals if v]
    n = max(len(filled), 1)
    dates = [d for d in (parse_date(v) for v in filled) if d]
    lens = Counter(len(v) for v in filled)
    return {
        "fill": len(filled) / max(len(vals), 1),
        "unique": len(set(filled)) / n,
        "date": len(dates) / n,
        "past": sum(1 for d in dates if d <= today()) / max(len(dates), 1),
        "median_date": statistics.median_low(dates) if dates else None,
        "idlike": sum(1 for v in filled if re.fullmatch(r"[A-Za-z0-9\-]{4,30}", v
        "digits": sum(1 for v in filled if v.isdigit()) / n,
        "len_consistent": (lens.most_common(1)[0][1] / n) if lens else 0.0,
        "distinct": len(set(filled)),
    }


def header_score(header, aliases):
    """1.0 for an exact alias match, else best fuzzy ratio if >= 0.75, else 0."""
    h = norm(header)
    if not h:
        return 0.0, None
    best, best_alias = 0.0, None
    for a in aliases:
        if h == a:
            return 1.0, a
        r = difflib.SequenceMatcher(None, h, a).ratio()
        if r > best:
            best, best_alias = r, a
    return (best, best_alias) if best >= 0.75 else (0.0, None)


def value_score(field, p, earliest_date_col):
```

```
    if field == "account_number":
        return (0.4 * p["unique"] + 0.3 * p["idlike"] + 0.3 * p["len_consistent"]
    if field == "chargeoff_date":
        if p["date"] <= 0.8:
            return 0.0
        base = p["date"] * p["past"]
        # Charge-off precedes placement/load dates: favour the earliest date colu
        return base if earliest_date_col else base * 0.8
    if field == "client_ref":
        return (0.5 * p["unique"] + 0.5 * p["idlike"]) * (1 - p["date"])
    # client_name: low-cardinality text
    return 1.0 if (p["distinct"] <= 5 and p["date"] < 0.1 and p["digits"] < 0.1 a


def _reason(field, hs, alias, p, artiva, earliest, n_date_cols):
    bits = []
    if hs == 1.0:
        bits.append(f"header matches '{alias}'")
    elif hs > 0:
        bits.append(f"header similar to '{alias}'")
    else:
        bits.append("header not recognised")
    if field in ("account_number", "client_ref"):
        bits.append(f"{p['unique']:.0%} unique")
    if field == "chargeoff_date":
        bits.append(f"{p['date']:.0%} valid dates")
        if n_date_cols > 1 and earliest:
            bits.append(f"earliest of {n_date_cols} date columns")
    if field == "client_name" and p["distinct"] <= 5:
        bits.append("same value(s) on every row")
    if artiva is not None:
        bits.append(f"{artiva:.0%} found in Artiva")
    return " · ".join(bits)


def guess_mapping(headers, columns, synonyms=None, artiva_rates=None):
    """Return {field: {col, header, score, conf, reason, evidence, date_format?}}

    synonyms:     {field: [normalized aliases]} (defaults to SEED_SYNONYMS)
    artiva_rates: {(field, col_index): rate 0..1} from artiva_check (optional)
    """
    synonyms = synonyms or SEED_SYNONYMS
    artiva_rates = artiva_rates or {}
    profiles = [profile_column(c) for c in columns]

    date_cols = [i for i, p in enumerate(profiles) if p["date"] > 0.8 and p["medi
```

```
earliest = min(date_cols, key=lambda i: profiles[i]["median_date"]) if date_c

cands = []
for f in FIELDS:
    for i, h in enumerate(headers):
        p = profiles[i]
        hs, alias = header_score(h, synonyms.get(f, []))
        vs = value_score(f, p, earliest_date_col=(i == earliest))
        ar = artiva_rates.get((f, i))
        if ar is None:
            score = 0.6 * hs + 0.4 * vs
        else:
            score = 0.35 * hs + 0.25 * vs + 0.40 * ar
        cands.append({
            "field": f, "col": i, "header": h, "score": round(score, 3),
            "evidence": {"header_score": round(hs, 2), "value_score": round(v
                         "artiva_rate": ar},
            "reason": _reason(f, hs, alias, p, ar, i == earliest, len(date_co
        })

cands.sort(key=lambda c: c["score"], reverse=True)
result, used = {}, set()
for c in cands:
    f, i = c["field"], c["col"]
    if f in result or i in used or c["score"] < 0.3:
        continue
    runner_up = max((x["score"] for x in cands if x["field"] == f and x["col"
    ar = c["evidence"]["artiva_rate"]
    if f == "account_number" and ar is not None and ar >= 0.9:
        conf = "HIGH"
    elif c["score"] >= 0.75 and c["score"] - runner_up >= 0.15:
        conf = "HIGH"
    elif c["score"] >= 0.5:
        conf = "MEDIUM"
    else:
        conf = "LOW"
    c["conf"] = conf
    c["evidence"]["runner_up"] = round(runner_up, 3)
    if f == "chargeoff_date":
        code, rate = detect_date_format(columns[i])
        c["date_format"] = code
        c["evidence"]["date_parse_rate"] = round(rate, 3)
    result[f] = c
    used.add(i)
return result
```

```
def overall_confidence(mapping):
    """HIGH only if both required fields are HIGH."""
    confs = [mapping.get(f, {}).get("conf", "NONE") for f in ("account_number", "
    if all(c == "HIGH" for c in confs):
        return "HIGH"
    if "NONE" in confs or "LOW" in confs:
        return "LOW"
    return "MEDIUM"


def to_field_maps(mapping, client_source="assigned_client"):
    """Convert a guess into the field_maps JSON stored on a format version."""
    fm = {}
    for f in ("account_number", "chargeoff_date", "client_ref"):
        if f in mapping:
            fm[f] = {"source": "header", "header": mapping[f]["header"], "index_f
    if client_source == "file_column" and "client_name" in mapping:
        fm["client_name"] = {"source": "header", "header": mapping["client_name"]
    else:
        fm["client_name"] = {"source": "assigned_client"}
    return fm
```

B.4 `placement_intake/parser.py`

> 🟢 **DROP-IN · TESTED**

```
"""Generic, config-driven parser for flat placement files + row validation. No DB

parse_config example:
  {"delimiter": "|", "quotechar": "\"", "encoding": "utf-8-sig", "header_row": 0,
   "skip_footer_rows": 1, "footer_marker_regex": "^(TRL|TRAILER|TOTAL)",
   "date_formats": ["%m/%d/%Y"]}

field_maps example:
  {"account_number": {"source": "header", "header": "Acct No", "aliases": [], "in
   "chargeoff_date": {"source": "header", "header": "C/O Date"},
   "client_name":    {"source": "assigned_client"},
   "client_ref":     {"source": "header", "header": "Client Ref"}}
"""
import csv
import os
import re
from collections import deque
```

```
from .common import DATE_CODES, norm, parse_date, today


class MappingError(Exception):
    """A mapped header is missing from the file: fail the file, never shift colum


def resolve_columns(header, field_maps):
    """Return {field: column_index} for header/index sources."""
    lookup = {}
    for i, h in enumerate(header):
        lookup.setdefault(norm(h), i)
    cols, missing = {}, []
    for field, m in field_maps.items():
        src = m.get("source")
        if src == "header":
            names = [m.get("header", "")] + list(m.get("aliases", []))
            idx = next((lookup[norm(n)] for n in names if norm(n) in lookup), Non
            if idx is None:
                missing.append(f"{field} (header '{m.get('header')}')")
            else:
                cols[field] = idx
        elif src == "index":
            idx = int(m["index"])
            if idx >= len(header):
                missing.append(f"{field} (column {idx + 1})")
            else:
                cols[field] = idx
    if missing:
        raise MappingError("Mapped column(s) not found in file: " + ", ".join(mis
    return cols


def _is_footer(row, marker_re):
    if not row or not any(c.strip() for c in row):
        return True
    return bool(marker_re and marker_re.search(row[0].strip()))


def iter_records(path, parse_config, field_maps, assigned_client_name=None):
    """Yield (row_num, record_dict, col_count_ok) for every data row.

    row_num  = 1-based line number in the file (header = header_row + 1).
    record["account_seq"] = 1-based position of this account among the data recor
                (1st account = 1). Counts every data record, rejected ones too, so
                matches "the 10th account in the placement file". Footer/blank lin
```

```
    """
    cfg = parse_config
    header_row = int(cfg.get("header_row", 0))
    skip_footer = int(cfg.get("skip_footer_rows", 0))
    marker = cfg.get("footer_marker_regex")
    marker_re = re.compile(marker, re.I) if marker else None
    fname = os.path.basename(path)

    with open(path, "r", encoding=cfg.get("encoding", "utf-8-sig"), errors="repla
        reader = csv.reader(fh, delimiter=cfg["delimiter"], quotechar=cfg.get("qu
        for _ in range(header_row):
            next(reader, None)
        header = next(reader, None)
        if header is None:
            return
        cols = resolve_columns(header, field_maps)
        width = len(header)

        # Buffer so the last N rows (footer/trailer) can be dropped while streami
        buf = deque()
        row_num = header_row + 1
        seq = 0
        for row in reader:
            row_num += 1
            buf.append((row_num, row))
            if len(buf) > skip_footer:
                n, r = buf.popleft()
                if _is_footer(r, marker_re):
                    continue
                seq += 1
                rec = _extract(r, cols, field_maps, fname, assigned_client_name)
                rec["account_seq"] = seq
                yield n, rec, len(r) == width


def _extract(row, cols, field_maps, fname, assigned_client_name):
    rec = {}
    for field, m in field_maps.items():
        src = m.get("source")
        if src in ("header", "index"):
            i = cols[field]
            rec[field] = row[i].strip() if i < len(row) else ""
        elif src == "constant":
            rec[field] = m.get("value", "")
        elif src == "assigned_client":
            rec[field] = assigned_client_name or ""
```

```
        elif src == "filename_regex":
            mt = re.search(m["pattern"], fname)
            rec[field] = mt.group(m.get("group", 1)) if mt else ""
        else:
            rec[field] = ""
    return rec


def validate(records, date_formats=None):
    """Validate an iterable of (row_num, rec, col_ok).

    Yields ("ok", row_num, clean_record) or ("error", row_num, field, code, accou
    A row with any error is not loaded. Duplicate accounts keep the first occurre
    """
    codes = date_formats or DATE_CODES
    seen = set()
    now = today()
    for row_num, rec, col_ok in records:
        errors = []
        if not col_ok:
            errors.append(("row", "col_count_mismatch"))
        acct = rec.get("account_number", "").strip()
        if not acct:
            errors.append(("account_number", "blank_account"))
        raw_co = rec.get("chargeoff_date", "").strip()
        co = parse_date(raw_co, codes) if raw_co else None
        if raw_co and co is None:
            errors.append(("chargeoff_date", "bad_date"))
        elif not raw_co:
            errors.append(("chargeoff_date", "blank_date"))
        elif co > now:
            errors.append(("chargeoff_date", "future_date"))
        if acct and acct in seen:
            errors.append(("account_number", "dup_in_file"))
        if errors:
            for field, code in errors:
                yield ("error", row_num, field, code, rec.get("account_seq"))
            continue
        seen.add(acct)
        yield ("ok", row_num, {
            "account_seq": rec.get("account_seq"),
            "account_number": acct,
            "chargeoff_date": co,
            "raw_chargeoff": raw_co,
            "client_name": rec.get("client_name", "").strip() or None,
            "client_ref": rec.get("client_ref", "").strip() or None,
```

```
        })
```

B.5 `placement_intake/template_io.py`

> 🟢 **DROP-IN · TESTED**

```
"""Option 2: Excel template - build (blank / pre-filled), read, and validate rows

Requires openpyxl. No DB access here: callers pass in clients/prods/segments and
validated rows; services.py persists them.
"""
import io
import os
import re
from dataclasses import dataclass, field

from openpyxl import Workbook, load_workbook
from openpyxl.comments import Comment
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter
from openpyxl.worksheet.datavalidation import DataValidation

from .common import (DATE_FORMATS, DATE_LABEL_BY_CODE, DELIMITERS, DELIMITER_LABE
                     norm, parse_date, signature, wildcard_to_regex)
from .discovery import read_sample

TEMPLATE_VERSION = "PI-TEMPLATE-1"
MAX_ROWS = 1000

# (key, header, required, width, help)
COLUMNS = [
    ("row_id", "Row ID", False, 8, "Auto-numbered. Do not edit."),
    ("prod", "Prod *", True, 12, "Pick from the list."),
    ("segment", "Segment *", True, 14, "Pick from the list."),
    ("folder_path", "Folder Path *", True, 48, r"Full path, e.g. \\fs01\Placement
    ("file_pattern", "File Name Pattern *", True, 24, "Use * for any text, e.g. A
    ("file_type", "File Type *", True, 18, "Flat (with header) for now."),
    ("delimiter", "Delimiter *", True, 12, "Character between columns."),
    ("header_row", "Header Row Number", False, 10, "Line number of the header. De
    ("footer_rows", "Footer Rows to Skip", False, 10, "Trailer lines at the end.
    ("acct_header", "Account Number Header *", True, 26, "Header text exactly as
    ("co_header", "Charge-off Date Header *", True, 26, "Header text exactly as i
    ("date_format", "Date Format *", True, 20, "How the charge-off date is writte
    ("ref_header", "Client Reference Header", False, 24, "Leave blank if not in t
    ("client_source", "Client Name Source *", True, 18, "Assign client, or From c
```

```
    ("client_code", "Client Code", False, 26, "Required when source is Assign cli
    ("client_header", "Client Name Header", False, 22, "Required when source is F
    ("notes", "Notes", False, 30, "Optional."),
]
KEYS = [c[0] for c in COLUMNS]
FILE_TYPES = ["Flat (with header)", "Fixed width"]
CLIENT_SOURCES = ["Assign client", "From column"]

REQ_FILL = PatternFill("solid", fgColor="FFF4CC")
HDR_FILL = PatternFill("solid", fgColor="1F3A5F")
ERR_FILL = PatternFill("solid", fgColor="F8D7DA")
OK_FILL = PatternFill("solid", fgColor="D4EDDA")


def build_template(prods, segments, clients, prefill_rows=None):
    """Return .xlsx bytes.

    clients: list of {"client_code", "client_name", "prod"}
    prefill_rows: list of dicts keyed by KEYS (from a discovery run), optional
    """
    wb = Workbook()
    _instructions_sheet(wb.active)

    ws = wb.create_sheet("File Types")
    for ci, (key, header, req, width, help_) in enumerate(COLUMNS, start=1):
        cell = ws.cell(row=1, column=ci, value=header)
        cell.font = Font(bold=True, color="FFFFFF")
        cell.fill = HDR_FILL
        cell.alignment = Alignment(wrap_text=True, vertical="center")
        cell.comment = Comment(help_, "iResolve")
        ws.column_dimensions[get_column_letter(ci)].width = width
    ws.row_dimensions[1].height = 34
    ws.freeze_panes = "B2"

    lists = wb.create_sheet("Lists")
    client_labels = [f"{c['client_code']} - {c['client_name']}" for c in clients]
    list_cols = {
        "prod": prods, "segment": segments, "file_type": FILE_TYPES,
        "delimiter": list(DELIMITERS), "date_format": list(DATE_FORMATS),
        "client_source": CLIENT_SOURCES, "client_code": client_labels,
    }
    for li, (key, values) in enumerate(list_cols.items(), start=1):
        lists.cell(row=1, column=li, value=key)
        for ri, v in enumerate(values, start=2):
            lists.cell(row=ri, column=li, value=v)
        letter = get_column_letter(li)
```

```
            ref = f"Lists!${letter}$2:${letter}${max(len(values) + 1, 2)}"
            dv = DataValidation(type="list", formula1=f"={ref}", allow_blank=True,
                                showErrorMessage=True, errorTitle="Pick from the list
                                error="Choose a value from the dropdown.")
            col_letter = get_column_letter(KEYS.index(key) + 1)
            dv.add(f"{col_letter}2:{col_letter}{MAX_ROWS + 1}")
            ws.add_data_validation(dv)
        lists.cell(row=1, column=20, value="template_version")
        lists.cell(row=2, column=20, value=TEMPLATE_VERSION)
        lists.sheet_state = "hidden"

        for r in range(2, MAX_ROWS + 2):
            ws.cell(row=r, column=1, value=r - 1)
            for ci, (_, _, req, _, _) in enumerate(COLUMNS, start=1):
                if req:
                    ws.cell(row=r, column=ci).fill = REQ_FILL

        for ri, row in enumerate(prefill_rows or [], start=2):
            for ci, key in enumerate(KEYS, start=1):
                if key != "row_id" and row.get(key) not in (None, ""):
                    ws.cell(row=ri, column=ci, value=row[key])

        wb.active = wb.sheetnames.index("File Types")
        out = io.BytesIO()
        wb.save(out)
        return out.getvalue()


    def _instructions_sheet(ws):
        ws.title = "Instructions"
        lines = [
            ("How to fill this template", True),
            ("1. Go to the 'File Types' sheet. Fill ONE row for each type of file in
            ("2. Yellow columns are required. Hover a column header to see help.", Fa
            ("3. Header names must match the file exactly (spaces and capitals do not
            ("4. If one folder holds files for two clients, add two rows with differe
            ("5. Save and upload the file in iResolve > Placement Intake > Set Up For
            ("6. iResolve checks every row against a real file. Nothing is saved unti
            ("", False),
            ("Example row", True),
            ("Prod: attprod | Segment: Telecom | Folder Path: \\\\fs01\\Placements\\T
             "File Name Pattern: ATT_PLC_*.txt | File Type: Flat (with header) | Deli
             "Account Number Header: Acct No | Charge-off Date Header: C/O Date | Dat
             "Client Reference Header: Client Ref | Client Name Source: Assign client
        ]
        for i, (text, bold) in enumerate(lines, start=1):
```

```
        c = ws.cell(row=i, column=1, value=text)
        c.font = Font(bold=bold, size=13 if bold else 11)
    ws.column_dimensions["A"].width = 140


# ------------------------------------------------------------------ reading + vali

@dataclass
class RowResult:
    row_id: int
    values: dict
    status: str = "ready"          # ready | update | error | blank
    messages: list = field(default_factory=list)
    fixes: list = field(default_factory=list)   # [{"field","value","label"}]
    resolved: dict = field(default_factory=dict)  # data needed to save the forma


def read_template(data: bytes):
    """Return list of value dicts (blank rows dropped). Raises ValueError on wron
    wb = load_workbook(io.BytesIO(data), data_only=True)
    if "File Types" not in wb.sheetnames or "Lists" not in wb.sheetnames:
        raise ValueError("This is not a Placement Intake template. Download a new
    if wb["Lists"].cell(row=2, column=20).value != TEMPLATE_VERSION:
        raise ValueError("This template is out of date. Download a new one and co
    ws = wb["File Types"]
    rows = []
    for r in ws.iter_rows(min_row=2, max_row=MAX_ROWS + 1, values_only=True):
        vals = {k: (str(v).strip() if v is not None else "") for k, v in zip(KEYS
        if not any(vals[k] for k in KEYS if k != "row_id"):
            continue
        rows.append(vals)
    return rows


def _closest(name, headers):
    import difflib
    by_norm = {norm(h): h for h in headers}
    hit = difflib.get_close_matches(norm(name), list(by_norm), n=1, cutoff=0.6)
    return by_norm[hit[0]] if hit else None


def validate_row(vals, ctx):
    """Validate one template row against the file system and reference data.

    ctx = {
      "prods": set, "segments": set, "roots": [str],
```

```
  "clients": {client_code: {"id","client_name","prod"}},
  "existing": {(folder_norm, pattern_regex): format_id}   # for 'update' stat
  "sample_rows": int
}
"""
res = RowResult(row_id=int(float(vals.get("row_id") or 0)), values=vals)
err = res.messages.append

# 1. Structure / lists
for key, header, req, *_ in COLUMNS:
    if req and not vals.get(key):
        err(f"'{header.rstrip(' *')}' is required.")
if vals["prod"] and vals["prod"] not in ctx["prods"]:
    err(f"Prod '{vals['prod']}' is not in the list.")
if vals["segment"] and vals["segment"] not in ctx["segments"]:
    err(f"Segment '{vals['segment']}' is not in the list.")
if vals["file_type"] == "Fixed width":
    err("Fixed width files are not supported yet. Remove this row for now.")
if vals["delimiter"] and vals["delimiter"] not in DELIMITERS:
    err(f"Delimiter '{vals['delimiter']}' is not in the list.")
if vals["date_format"] and vals["date_format"] not in DATE_FORMATS:
    err(f"Date format '{vals['date_format']}' is not in the list.")

client = None
if vals["client_source"] == "Assign client":
    code = vals["client_code"].split(" - ")[0].strip()
    client = ctx["clients"].get(code)
    if not code:
        err("Client Code is required when Client Name Source is 'Assign clien
    elif not client:
        err(f"Client code '{code}' was not found.")
    elif vals["prod"] and client["prod"] != vals["prod"]:
        err(f"Client '{code}' belongs to {client['prod']}, not {vals['prod']}
elif vals["client_source"] == "From column" and not vals["client_header"]:
    err("Client Name Header is required when Client Name Source is 'From colu

def int_or(v, default):
    try:
        return int(float(v)) if v else default
    except ValueError:
        err(f"'{v}' is not a whole number.")
        return default

header_row = int_or(vals["header_row"], 1)
footer_rows = int_or(vals["footer_rows"], 0)
if res.messages:
```

```
    res.status = "error"
    return res

# 2. Folder and files
folder = vals["folder_path"].rstrip("\\/")
if ctx.get("roots") and not any(norm(folder).startswith(norm(r)) for r in ctx
    err("Folder is outside the watched placement folders (see Settings).")
elif not os.path.isdir(folder):
    err("Folder not found, or iResolve cannot access it.")
if res.messages:
    res.status = "error"
    return res
rx = re.compile(wildcard_to_regex(vals["file_pattern"]))
matches = [os.path.join(folder, f) for f in os.listdir(folder)
           if os.path.isfile(os.path.join(folder, f)) and rx.match(f)]
if not matches:
    err(f"No files in this folder match '{vals['file_pattern']}'.")
    res.status = "error"
    return res
newest = max(matches, key=os.path.getmtime)

# 3. Layout
delim = DELIMITERS[vals["delimiter"]]
info, status = read_sample(newest, ctx.get("sample_rows", 500), delimiter=del
                           header_row=header_row - 1)
if not info:
    err(f"Could not read the header of {os.path.basename(newest)} ({status}).
    res.status = "error"
    return res
headers = info["headers"]
if len(headers) < 2:
    err(f"The {vals['delimiter']} delimiter does not split the header row. Ch
    res.status = "error"
    return res
hmap = {norm(h): i for i, h in enumerate(headers)}

def check_header(key, label):
    name = vals.get(key)
    if not name:
        return None
    if norm(name) in hmap:
        return hmap[norm(name)]
    near = _closest(name, headers)
    if near:
        err(f"{label} header '{name}' not found. Did you mean '{near}'?")
        res.fixes.append({"field": key, "value": near, "label": f"Use {near}"
```

```
    else:
        err(f"{label} header '{name}' not found in {os.path.basename(newest)}
    return None

acct_i = check_header("acct_header", "Account Number")
co_i = check_header("co_header", "Charge-off Date")
check_header("ref_header", "Client Reference")
if vals["client_source"] == "From column":
    check_header("client_header", "Client Name")
if acct_i is not None and acct_i == co_i:
    err("Account Number and Charge-off Date point to the same column.")

# 4. Data checks on the sample
if co_i is not None and vals["date_format"] in DATE_FORMATS:
    code = DATE_FORMATS[vals["date_format"]]
    vals_co = [v for v in info["columns"][co_i] if v.strip()]
    ok = sum(1 for v in vals_co if parse_date(v, [code]))
    rate = ok / max(len(vals_co), 1)
    if rate < 0.95:
        from .common import detect_date_format
        best, best_rate = detect_date_format(vals_co)
        msg = f"Date format {vals['date_format']} read only {rate:.0%} of the
        if best and best_rate >= 0.95:
            label = DATE_LABEL_BY_CODE[best]
            msg += f" They look like {label}."
            res.fixes.append({"field": "date_format", "value": label, "label"
        err(msg)
if acct_i is not None:
    col = info["columns"][acct_i]
    filled = sum(1 for v in col if v.strip()) / max(len(col), 1)
    if filled < 0.99:
        err(f"Account Number column is blank on {1 - filled:.0%} of sample ro

if res.messages:
    res.status = "error"
    return res

# 5. Resolved config for saving
pattern_rx = wildcard_to_regex(vals["file_pattern"])
field_maps = {
    "account_number": {"source": "header", "header": headers[acct_i], "index_
    "chargeoff_date": {"source": "header", "header": headers[co_i]},
}
if vals["ref_header"]:
    field_maps["client_ref"] = {"source": "header", "header": vals["ref_heade
if vals["client_source"] == "From column":
```

```
            field_maps["client_name"] = {"source": "header", "header": vals["client_h
        else:
            field_maps["client_name"] = {"source": "assigned_client"}
        res.resolved = {
            "signature": signature(headers),
            "headers": headers,
            "example_file": newest,
            "matching_files": len(matches),
            "folder_path": folder,
            "filename_regex": pattern_rx,
            "prod": vals["prod"],
            "segment": vals["segment"],
            "client_id": client["id"] if client else None,
            "client_source": "assigned" if client else "file_column",
            "parse_config": {
                "delimiter": delim, "quotechar": '"', "encoding": "utf-8-sig",
                "header_row": header_row - 1, "skip_footer_rows": footer_rows,
                "date_formats": [DATE_FORMATS[vals["date_format"]]],
            },
            "field_maps": field_maps,
            "notes": vals["notes"],
        }
        key = (norm(folder), pattern_rx)
        res.status = "update" if key in ctx.get("existing", {}) else "ready"
        return res


    def apply_fix(vals, field_key, value):
        """Return a copy of row values with one field changed (used by the 'fix in pl
        out = dict(vals)
        out[field_key] = value
        return out


    def errors_workbook(original: bytes, results):
        """Return the uploaded workbook with Status and Message columns and coloured
        wb = load_workbook(io.BytesIO(original))
        ws = wb["File Types"]
        sc, mc = len(COLUMNS) + 1, len(COLUMNS) + 2
        for c, title in ((sc, "Status"), (mc, "Message")):
            cell = ws.cell(row=1, column=c, value=title)
            cell.font = Font(bold=True, color="FFFFFF")
            cell.fill = HDR_FILL
        ws.column_dimensions[get_column_letter(mc)].width = 80
        by_id = {r.row_id: r for r in results}
        for r in range(2, ws.max_row + 1):
```

```
            rid = ws.cell(row=r, column=1).value
            res = by_id.get(int(rid)) if rid else None
            if not res:
                continue
            ws.cell(row=r, column=sc, value=res.status.upper())
            ws.cell(row=r, column=mc, value=" | ".join(res.messages) or "OK")
            fill = ERR_FILL if res.status == "error" else OK_FILL
            for c in range(1, mc + 1):
                ws.cell(row=r, column=c).fill = fill
    out = io.BytesIO()
    wb.save(out)
    return out.getvalue()


def prefill_rows_from_formats(formats, segment_defaults):
    """Build template rows from discovery formats (one row per format x folder).

    formats: list of {signature, headers, delimiter, folders: {path: count}, gues
    segment_defaults: {folder_path_norm: {"prod", "segment"}} (prefix match)
    """
    rows = []
    for f in formats:
        g = f.get("guess", {})
        for folder in f["folders"]:
            sd = next((v for k, v in segment_defaults.items() if norm(folder).sta
            ext = os.path.splitext(f.get("example_file", ""))[1] or ".*"
            rows.append({
                "prod": sd.get("prod", ""), "segment": sd.get("segment", ""),
                "folder_path": folder, "file_pattern": f"*{ext}",
                "file_type": "Flat (with header)",
                "delimiter": DELIMITER_LABEL.get(f["delimiter"], ""),
                "header_row": 1, "footer_rows": 0,
                "acct_header": g.get("account_number", {}).get("header", ""),
                "co_header": g.get("chargeoff_date", {}).get("header", ""),
                "date_format": DATE_LABEL_BY_CODE.get(g.get("chargeoff_date", {})
                "ref_header": g.get("client_ref", {}).get("header", ""),
                "client_source": "Assign client",
                "notes": "All headers: " + " | ".join(f["headers"]),
            })
    return rows
```

B.6 `placement_intake/run_log.py`

> 🟢 **DROP-IN · TESTED**

```
"""One plain-text log file per run. Stdlib only, no Django.

    with RunLog(log_dir, run_id=41, mode="process", user="kyash") as log:
        log.info("Roots: \\\\fs01\\Placements")
        log.file(path, "processed", rows_ok=9980, rows_error=20, client="ATT Wire
        log.warn("Folder not reachable: ...")
        log.summary({"files": 42, "accounts": 318220})

File name: pi_run000041_process_20261022_103000.log
NEVER log account numbers, names, dates of birth or any row values - only paths,
"""
import os
import time
import traceback
from datetime import datetime

LEVELS = ("INFO", "WARN", "ERROR")


class RunLog:
    def __init__(self, log_dir, run_id, mode, user=""):
        self.run_id, self.mode, self.user = run_id, mode, user
        self.started = time.monotonic()
        self.counts = {"INFO": 0, "WARN": 0, "ERROR": 0}
        self.path = None
        self._fh = None
        if log_dir:
            os.makedirs(log_dir, exist_ok=True)
            name = f"pi_run{run_id:06d}_{mode}_{datetime.now():%Y%m%d_%H%M%S}.log
            self.path = os.path.join(log_dir, name)
            self._fh = open(self.path, "a", encoding="utf-8", buffering=1)  # lin

    # -- context manager: always writes a footer, even if the run crashes
    def __enter__(self):
        self._write("INFO", f"=== Run #{self.run_id} ({self.mode}) started by {se
        return self

    def __exit__(self, exc_type, exc, tb):
        if exc_type:
            self._write("ERROR", f"Run stopped with an error: {exc_type.__name__}
            for line in traceback.format_exception(exc_type, exc, tb)[-6:]:
                self._write("ERROR", "  " + line.rstrip(), count=False)
        secs = time.monotonic() - self.started
        status = "FAILED" if exc_type else "FINISHED"
        self._write("INFO", f"=== Run #{self.run_id} {status} in {_dur(secs)} · "
                            f"{self.counts['WARN']} warnings · {self.counts['ERRO
```

```
    self.close()
    return False  # never swallow the exception

def _write(self, level, msg, count=True):
    if count:
        self.counts[level] = self.counts.get(level, 0) + 1
    if self._fh:
        self._fh.write(f"{datetime.now():%Y-%m-%d %H:%M:%S} {level:<5} {msg}\

def info(self, msg):
    self._write("INFO", msg)

def warn(self, msg):
    self._write("WARN", msg)

def error(self, msg):
    self._write("ERROR", msg)

def section(self, title):
    """Visual separator, e.g. one per file."""
    self._write("INFO", f"--- {title} " + "-" * max(0, 70 - len(title)), coun

def kv(self, title, data, level="INFO"):
    """One line: 'title: k=v · k=v'. Dict values are flattened one level."""
    parts = []
    for k, v in data.items():
        if isinstance(v, dict):
            v = ", ".join(f"{a}={b}" for a, b in v.items())
        elif isinstance(v, (list, tuple)):
            v = ", ".join(str(x) for x in v)
        parts.append(f"{k}={v}")
    self._write(level, f"{title}: " + " · ".join(parts), count=level != "INFO

def detail(self, msg):
    """Indented detail line under the current file/item (not counted)."""
    self._write("INFO", "    " + msg, count=False)

def file(self, path, status, **details):
    """One line per file. details: rows_ok, rows_error, client, file_type, me
    level = "ERROR" if status == "failed" else "WARN" if status in ("partial"
    parts = [f"[{status.upper()}]", path]
    for k in ("file_type", "client", "rows_ok", "rows_error"):
        if details.get(k) not in (None, ""):
            parts.append(f"{k}={details[k]}")
    if details.get("error_codes"):
        parts.append("errors=" + ",".join(f"{c}:{n}" for c, n in sorted(detai
```

```
        if details.get("message"):
            parts.append(f"- {details['message']}")
        self._write(level, " ".join(str(p) for p in parts))

    def summary(self, stats):
        self._write("INFO", "Summary: " + " · ".join(f"{k}={v}" for k, v in stats
                                                     if not isinstance(v, dict)))
        for k, v in stats.items():
            if isinstance(v, dict):
                self._write("INFO", f"  {k}: " + ", ".join(f"{a}={b}" for a, b in

    def close(self):
        if self._fh:
            self._fh.close()
            self._fh = None


def _dur(secs):
    m, s = divmod(int(secs), 60)
    h, m = divmod(m, 60)
    return f"{h}h {m}m {s}s" if h else f"{m}m {s}s" if m else f"{s}s"


def prune_logs(log_dir, keep_days):
    """Delete pi_run*.log files older than keep_days. Returns number deleted."""
    if not log_dir or not keep_days or not os.path.isdir(log_dir):
        return 0
    cutoff = time.time() - keep_days * 86400
    n = 0
    for name in os.listdir(log_dir):
        p = os.path.join(log_dir, name)
        if name.startswith("pi_run") and name.endswith(".log") and os.path.getmti
            os.remove(p)
            n += 1
    return n


def tail(path, max_lines=500):
    """Last N lines of a log (for the in-app viewer)."""
    if not path or not os.path.exists(path):
        return []
    with open(path, encoding="utf-8", errors="replace") as fh:
        lines = fh.readlines()
    return [ln.rstrip("\n") for ln in lines[-max_lines:]]
```

B.7 `placement_intake/cli_discover.py`

> 🟢 **DROP-IN · TESTED**

```
"""Standalone dry run (no Django, no DB). Run BEFORE building anything to size th

    python -m placement_intake.cli_discover --root "\\\\fs01\\Placements" --out .

Writes <out>/formats.csv (one row per distinct file layout, with suggested mappin
<out>/files.csv (one row per file, with its layout signature or why it was skippe
Never writes account values - only headers and column statistics.
"""
import argparse
import csv
import os

from .automap import guess_mapping, overall_confidence
from .common import DATE_LABEL_BY_CODE, FIELDS
from .discovery import scan, status_counts


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--root", required=True, action="append", help="repeat for se
    ap.add_argument("--out", default="discovery")
    ap.add_argument("--sample-rows", type=int, default=500)
    ap.add_argument("--since-days", type=int)
    a = ap.parse_args()
    os.makedirs(a.out, exist_ok=True)

    folder_errors = []
    result = scan(a.root, a.since_days, a.sample_rows,
                  progress=lambda f, n: print(f"\r{f} files, {n} layouts", end=""
                  on_error=lambda p, m: folder_errors.append((p, m)))
    print()
    for p, m in folder_errors:
        print(f"WARNING: folder not scanned: {p} - {m}")

    with open(os.path.join(a.out, "formats.csv"), "w", newline="", encoding="utf-
        w = csv.writer(fh)
        hdr = ["signature", "file_count", "folders", "delimiter", "column_count",
               "latest_example", "first_seen", "last_seen", "overall_confidence",
        for f in FIELDS:
            hdr += [f"{f}_header", f"{f}_conf", f"{f}_reason"]
        w.writerow(hdr)
        for sig, f in sorted(result["formats"].items(), key=lambda kv: -kv[1]["fi
```

```
            latest = f["latest"]
            g = guess_mapping(f["headers"], latest["info"]["columns"])
            row = [sig, f["file_count"], " ; ".join(f"{k} ({v})" for k, v in f["f
                   repr(f["delimiter"]), len(f["headers"]), " | ".join(f["headers
                   f["first_seen"].isoformat(timespec="seconds"), f["last_seen"].
                   overall_confidence(g),
                   DATE_LABEL_BY_CODE.get(g.get("chargeoff_date", {}).get("date_f
            for fld in FIELDS:
                m = g.get(fld)
                row += [m["header"], m["conf"], m["reason"]] if m else ["", "NONE
            w.writerow(row)

    with open(os.path.join(a.out, "files.csv"), "w", newline="", encoding="utf-8"
        w = csv.writer(fh)
        w.writerow(["path", "folder", "modified", "signature", "status"])
        for f in result["files"]:
            w.writerow([f["path"], f["folder"], f["mtime"].isoformat(timespec="se
                        f["signature"] or "", f["status"]])

    print(f"files scanned: {len(result['files'])}   distinct layouts: {len(result
    for status, n in status_counts(result["files"]).most_common():
        print(f"  {status}: {n}")


if __name__ == "__main__":
    main()
```

B.8 `placement_intake/artiva_check.py`

> 🟡 **ADAPT · NOT RUN — fill ARTIVA_LOOKUP and the connection helper**

```
"""Optional ground-truth signal: how many of a column's values exist in Artiva.

Read-only, sampled, never logs values. If ARTIVA_LOOKUP has no entry for a prod,
every function returns {} and the auto-mapper falls back to header + value signal

CODEX: fill ARTIVA_LOOKUP with the real table/column names per prod and point
get_artiva_connection() at the existing pyodbc connection factory used by the ETL
"""
import logging
import random

from .common import parse_date

log = logging.getLogger(__name__)
```

```
# prod -> where client account numbers and charge-off dates live in Artiva
ARTIVA_LOOKUP = {
    # "attprod": {"table": "SQLUser.ACCOUNT", "acct_col": "CLIENT_ACCT_NO", "co_c
}
SAMPLE = 300
CHUNK = 100   # keep IN-lists small for Cache/IRIS


def get_artiva_connection(prod):
    """Return a pyodbc connection to the prod's Artiva instance (wire to existing
    raise NotImplementedError("Wire to the existing Artiva pyodbc connection help


def _chunks(seq, n):
    for i in range(0, len(seq), n):
        yield seq[i:i + n]


def candidate_rates(prod, mapping_candidates, columns):
    """Compute Artiva match rates for the top candidates.

    mapping_candidates: {"account_number": [col_idx, ...], "chargeoff_date": [col
    columns: list of value lists (sample)
    Returns {(field, col_idx): rate}
    """
    cfg = ARTIVA_LOOKUP.get(prod)
    if not cfg:
        return {}
    rates = {}
    try:
        conn = get_artiva_connection(prod)
    except Exception:  # noqa: BLE001
        log.warning("Artiva check skipped for %s: no connection", prod)
        return {}
    try:
        cur = conn.cursor()
        best_acct, best_rate, found = None, -1.0, {}
        for col in mapping_candidates.get("account_number", []):
            values = list({v.strip() for v in columns[col] if v.strip()})
            sample = random.sample(values, min(SAMPLE, len(values)))
            hits = {}
            for chunk in _chunks(sample, CHUNK):
                ph = ",".join("?" * len(chunk))
                cur.execute(f"SELECT {cfg['acct_col']}, {cfg['co_col']} FROM {cfg
                            f"WHERE {cfg['acct_col']} IN ({ph})", chunk)
```

```
                for acct, co in cur.fetchall():
                    hits[str(acct).strip()] = co
            rate = len(hits) / max(len(sample), 1)
            rates[("account_number", col)] = rate
            if rate > best_rate:
                best_acct, best_rate, found = col, rate, hits

        # Charge-off: compare candidate date columns against Artiva for matched a
        if best_acct is not None and found:
            acct_vals = [v.strip() for v in columns[best_acct]]
            for col in mapping_candidates.get("chargeoff_date", []):
                same = total = 0
                for acct, raw in zip(acct_vals, columns[col]):
                    if acct in found and found[acct] is not None:
                        total += 1
                        d = parse_date(raw)
                        art = found[acct]
                        art = art.date() if hasattr(art, "date") else art
                        same += int(d is not None and d == art)
                rates[("chargeoff_date", col)] = same / total if total else 0.0
    finally:
        conn.close()
    log.info("Artiva check %s: %s", prod, {f"{k[0]}:{k[1]}": round(v, 2) for k, v
    return rates
```

B.9 `placement_intake/apps.py`

> 🟡 **ADAPT · NOT RUN**

```
from django.apps import AppConfig


class PlacementIntakeConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "placement_intake"
    verbose_name = "Placement Intake"
```

B.10 `placement_intake/models.py`

> 🟡 **ADAPT · NOT RUN — generate migrations; align naming with iResolve**

```
"""Django models. Table prefix pi_ = placement intake. Postgres (JSONField -> jso
from django.conf import settings
from django.db import models
```

```
class Setting(models.Model):
    """Key/value settings: roots (list), export_dir (str), export_format ("csv"|"
    auto_approve (bool), schedule_minutes (int), log_dir (str), log_retention_day
    key = models.CharField(max_length=64, primary_key=True)
    value = models.JSONField()

    class Meta:
        db_table = "pi_settings"

    @classmethod
    def get(cls, key, default=None):
        row = cls.objects.filter(key=key).first()
        return row.value if row else default


class Client(models.Model):
    client_code = models.CharField(max_length=32, unique=True)
    client_name = models.CharField(max_length=200)
    prod = models.CharField(max_length=32, db_index=True)
    active = models.BooleanField(default=True)

    class Meta:
        db_table = "pi_clients"
        ordering = ["prod", "client_name"]


class SegmentDefault(models.Model):
    folder_path = models.TextField(unique=True)
    prod = models.CharField(max_length=32)
    segment = models.CharField(max_length=64)
    date_format = models.CharField(max_length=32, blank=True, default="")
    active = models.BooleanField(default=True)

    class Meta:
        db_table = "pi_segment_defaults"


class HeaderSynonym(models.Model):
    field = models.CharField(max_length=32)
    alias_norm = models.CharField(max_length=200)
    source = models.CharField(max_length=16)  # seed | learned
    created_by = models.CharField(max_length=100, blank=True, default="")
    created_at = models.DateTimeField(auto_now_add=True)
```

```
    class Meta:
        db_table = "pi_header_synonyms"
        constraints = [models.UniqueConstraint(fields=["field", "alias_norm"], na


class Format(models.Model):
    STATUS = [("needs_review", "Needs review"), ("approved", "Approved"), ("retir
    signature = models.CharField(max_length=16, unique=True)
    format_type = models.CharField(max_length=16, default="flat")
    display_name = models.CharField(max_length=200, blank=True, default="")
    headers = models.JSONField()
    delimiter = models.CharField(max_length=4, blank=True, default="")
    status = models.CharField(max_length=16, choices=STATUS, db_index=True)
    suggestion = models.JSONField(null=True, blank=True)    # latest auto-map gue
    example_file = models.TextField(blank=True, default="")
    active_version = models.ForeignKey("FormatVersion", null=True, blank=True,
                                       on_delete=models.SET_NULL, related_name="+
    first_seen = models.DateTimeField(null=True)
    last_seen = models.DateTimeField(null=True)
    file_count = models.IntegerField(default=0)
    notes = models.TextField(blank=True, default="")

    class Meta:
        db_table = "pi_formats"


class FormatVersion(models.Model):
    format = models.ForeignKey(Format, on_delete=models.CASCADE, related_name="ve
    version = models.IntegerField()
    parse_config = models.JSONField()
    field_maps = models.JSONField()
    evidence = models.JSONField(null=True, blank=True)
    approval = models.CharField(max_length=16)  # human | auto | auto_drift | exc
    approved_by = models.CharField(max_length=100, blank=True, default="")
    approved_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = "pi_format_versions"
        constraints = [models.UniqueConstraint(fields=["format", "version"], name


class FormatFolder(models.Model):
    format = models.ForeignKey(Format, on_delete=models.CASCADE, related_name="fo
    folder_path = models.TextField()
    file_count = models.IntegerField(default=0)
```

```
    class Meta:
        db_table = "pi_format_folders"
        constraints = [models.UniqueConstraint(fields=["format", "folder_path"],


class FormatClient(models.Model):
    """Client per (format, folder, optional filename regex). Most specific match
    format = models.ForeignKey(Format, on_delete=models.CASCADE, related_name="cl
    folder_path = models.TextField()
    filename_regex = models.TextField(blank=True, default="")   # "" = whole fold
    file_pattern = models.CharField(max_length=200, blank=True, default="")  # wh
    client = models.ForeignKey(Client, null=True, blank=True, on_delete=models.PR
    client_source = models.CharField(max_length=16)  # assigned | file_column
    source = models.CharField(max_length=16)         # wizard | excel_template
    created_by = models.CharField(max_length=100, blank=True, default="")
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = "pi_format_clients"
        constraints = [models.UniqueConstraint(fields=["format", "folder_path", "
                                               name="pi_fc_uniq")]


class Run(models.Model):
    mode = models.CharField(max_length=16)          # discovery | process | repro
    status = models.CharField(max_length=16, default="queued")  # queued | runnin
    params = models.JSONField(null=True, blank=True)
    progress = models.JSONField(null=True, blank=True)  # {"files": n, "formats":
    stats = models.JSONField(null=True, blank=True)
    triggered_by = models.CharField(max_length=100, blank=True, default="")
    started_at = models.DateTimeField(auto_now_add=True)
    finished_at = models.DateTimeField(null=True)
    export_path = models.TextField(blank=True, default="")
    log_path = models.TextField(blank=True, default="")   # one .log file per run
    error = models.TextField(blank=True, default="")

    class Meta:
        db_table = "pi_runs"
        ordering = ["-id"]


class DiscoveryItem(models.Model):
    """Formats seen in one discovery run (wizard step 2)."""
    STATE = [("new", "New"), ("existing", "Already set up"), ("changed", "Layout
    run = models.ForeignKey(Run, on_delete=models.CASCADE, related_name="items")
    format = models.ForeignKey(Format, on_delete=models.CASCADE)
```

```
    file_count = models.IntegerField()
    folders = models.JSONField()      # {folder: count}
    state = models.CharField(max_length=16, choices=STATE)

    class Meta:
        db_table = "pi_discovery_items"
        constraints = [models.UniqueConstraint(fields=["run", "format"], name="pi


class DiscoveryFile(models.Model):
    run = models.ForeignKey(Run, on_delete=models.CASCADE, related_name="discover
    file_path = models.TextField()
    folder = models.TextField()
    signature = models.CharField(max_length=16, blank=True, default="")
    status = models.CharField(max_length=48)

    class Meta:
        db_table = "pi_discovery_files"


class PlacementFile(models.Model):
    STATUS = [("processed", "Processed"), ("partial", "Partial"), ("needs_review"
              ("headerless", "Parked"), ("failed", "Failed")]
    run = models.ForeignKey(Run, null=True, on_delete=models.SET_NULL)
    file_name = models.TextField()
    file_path = models.TextField()
    file_hash = models.CharField(max_length=64, unique=True)
    file_size = models.BigIntegerField(null=True)
    file_mtime = models.DateTimeField(null=True)
    prod = models.CharField(max_length=32, blank=True, default="", db_index=True)
    segment = models.CharField(max_length=64, blank=True, default="")
    signature = models.CharField(max_length=16, blank=True, default="", db_index=
    format_version = models.ForeignKey(FormatVersion, null=True, on_delete=models
    client = models.ForeignKey(Client, null=True, on_delete=models.PROTECT)
    status = models.CharField(max_length=16, choices=STATUS, db_index=True)
    rows_total = models.IntegerField(default=0)
    rows_ok = models.IntegerField(default=0)
    rows_error = models.IntegerField(default=0)
    message = models.TextField(blank=True, default="")
    processed_at = models.DateTimeField(null=True)

    class Meta:
        db_table = "pi_files"
        ordering = ["-id"]
```

```
class Account(models.Model):
    file = models.ForeignKey(PlacementFile, on_delete=models.CASCADE, related_nam
    account_seq = models.IntegerField(null=True)    # 1st, 2nd ... account in the
    row_num = models.IntegerField()                 # physical line number in the
    account_number = models.CharField(max_length=64, db_index=True)
    chargeoff_date = models.DateField(null=True)
    client_name = models.CharField(max_length=200, null=True)
    client_ref = models.CharField(max_length=100, null=True)
    raw_chargeoff = models.CharField(max_length=40, null=True)

    class Meta:
        db_table = "pi_accounts"


class RowError(models.Model):
    file = models.ForeignKey(PlacementFile, on_delete=models.CASCADE, related_nam
    account_seq = models.IntegerField(null=True)
    row_num = models.IntegerField()
    field = models.CharField(max_length=32)
    error_code = models.CharField(max_length=32)

    class Meta:
        db_table = "pi_row_errors"


class TemplateImport(models.Model):
    file_name = models.TextField()
    original = models.BinaryField()
    rows = models.JSONField()  # [{row_id, values, status, messages, fixes, resol
    status = models.CharField(max_length=16, default="validated")  # validated |
    uploaded_by = models.CharField(max_length=100, blank=True, default="")
    uploaded_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = "pi_template_imports"
```

B.11 `placement_intake/services.py`

> 🟡 **ADAPT · NOT RUN — all DB logic and run logging; keep behaviour**

```
"""Business logic that touches the database. Views and Celery tasks call only the
import csv
import logging
import os
import re
```

```
from contextlib import contextmanager
from datetime import datetime

from django.db import transaction
from django.utils import timezone

from . import template_io
from .artiva_check import candidate_rates
from .automap import SEED_SYNONYMS, guess_mapping, overall_confidence, to_field_m
from .common import (DATE_FORMATS, DATE_LABEL_BY_CODE, FIELDS, mask, norm, sha256
                     signature, wildcard_to_regex)
from .discovery import FLAT_EXTS, iter_files, read_sample, scan
from .models import (Account, Client, DiscoveryFile, DiscoveryItem, Format, Forma
                     FormatFolder, FormatVersion, HeaderSynonym, PlacementFile, R
                     SegmentDefault, Setting, TemplateImport)
from .parser import MappingError, iter_records, validate
from .run_log import RunLog, prune_logs

log = logging.getLogger(__name__)
SAMPLE_ROWS = 500
INSERT_BATCH = 5000


# ---------------------------------------------------------------- reference da

def seed_synonyms():
    for field, aliases in SEED_SYNONYMS.items():
        for a in aliases:
            HeaderSynonym.objects.get_or_create(field=field, alias_norm=norm(a),


def get_synonyms():
    out = {f: [] for f in FIELDS}
    for s in HeaderSynonym.objects.all():
        out.setdefault(s.field, []).append(s.alias_norm)
    return out


def learn_synonyms(field_maps, user):
    for field, m in field_maps.items():
        if m.get("source") == "header" and m.get("header"):
            HeaderSynonym.objects.get_or_create(
                field=field, alias_norm=norm(m["header"]),
                defaults={"source": "learned", "created_by": user})
```

```
def segment_for(path):
    """Longest matching folder prefix in SegmentDefault -> (prod, segment)."""
    p = norm(path)
    best = None
    for sd in SegmentDefault.objects.filter(active=True):
        k = norm(sd.folder_path)
        if p.startswith(k) and (best is None or len(k) > len(norm(best.folder_pat
            best = sd
    return (best.prod, best.segment) if best else ("", "")


def roots():
    return Setting.get("roots", [])


def log_dir():
    """Setting 'log_dir', else <BASE_DIR>/logs/placement_intake. A log is always
    from django.conf import settings as dj
    return Setting.get("log_dir") or os.path.join(str(getattr(dj, "BASE_DIR", "."


@contextmanager
def run_logged(run, user=""):
    """Open the run's log file, store its path on the run right away (so it can b
    while running or after a crash), mark the run running, and always write a foo
    rl = RunLog(log_dir(), run.id, run.mode, user or run.triggered_by)
    run.log_path = rl.path or ""
    run.status = "running"
    run.save(update_fields=["log_path", "status"])
    with rl:
        import platform
        rl.kv("Environment", {"host": platform.node(), "pid": os.getpid(), "pytho
        rl.kv("Run parameters", run.params or {})
        rl.kv("Settings", {k: Setting.get(k) for k in ("roots", "export_dir", "ex
                                                       "log_retention_days")})
        yield rl


def _error_codes(pf):
    from django.db.models import Count
    return {r["error_code"]: r["n"] for r in pf.row_errors.values("error_code").a


def log_file_result(log, pf):
    if not log:
        return
```

```
    log.file(pf.file_path, pf.status, rows_ok=pf.rows_ok, rows_error=pf.rows_erro
             client=pf.client.client_name if pf.client_id else "",
             file_type=pf.format_version.format.display_name if pf.format_version
             message=pf.message,
             error_codes=_error_codes(pf) if pf.status == "partial" else None)


# ------------------------------------------------------------------ discovery (O

def _display_name(fmt_folders, example_file):
    folder = max(fmt_folders, key=fmt_folders.get) if fmt_folders else ""
    leaf = os.path.basename(folder.rstrip("\\/")) or folder
    stem = re.sub(r"\d{6,}", "*", os.path.splitext(os.path.basename(example_file)
    return f"{leaf} - {stem}"


def run_discovery(run_id):
    run = Run.objects.get(pk=run_id)
    with run_logged(run) as log:
        return _run_discovery(run, log)


def _run_discovery(run, log):
    run_id = run.id
    params = run.params or {}
    log.info(f"Scan folders: {', '.join(params.get('roots') or roots())}")
    if params.get("since_days"):
        log.info(f"Only files changed in the last {params['since_days']} days")
    synonyms = get_synonyms()

    def progress(files, formats):
        Run.objects.filter(pk=run_id).update(progress={"files": files, "formats":

    result = scan(params.get("roots") or roots(), params.get("since_days"),
                  params.get("sample_rows", SAMPLE_ROWS), progress,
                  on_error=lambda path, msg: log.error(f"[FOLDER] {path} - {msg}"
    log.info(f"Read {len(result['files'])} files; found {len(result['formats'])}

    for f in result["files"]:
        if f["status"] != "ok":
            log.warn(f"[{f['status'].upper()}] {f['path']}")
    DiscoveryFile.objects.bulk_create([
        DiscoveryFile(run=run, file_path=f["path"], folder=f["folder"],
                      signature=f["signature"] or "", status=f["status"])
        for f in result["files"]], batch_size=2000)
```

```
counts = {"new": 0, "existing": 0, "changed": 0}
for sig, f in result["formats"].items():
    latest = f["latest"]
    prod, _ = segment_for(latest["folder"])
    guess = guess_mapping(f["headers"], latest["info"]["columns"], synonyms)
    rates = candidate_rates(prod, _top_candidates(guess, f["headers"], latest
                            latest["info"]["columns"])
    if rates:
        guess = guess_mapping(f["headers"], latest["info"]["columns"], synony
    suggestion = _suggestion_json(guess, f["delimiter"])

    fmt = Format.objects.filter(signature=sig).first()
    if fmt is None:
        fmt = Format.objects.create(
            signature=sig, headers=f["headers"], delimiter=f["delimiter"],
            status="needs_review", suggestion=suggestion, example_file=latest
            display_name=_display_name(dict(f["folders"]), latest["path"]),
            first_seen=timezone.make_aware(f["first_seen"]),
            last_seen=timezone.make_aware(f["last_seen"]), file_count=f["file
        state = "changed" if _folder_had_other_format(f["folders"], fmt) else
    else:
        fmt.suggestion = suggestion if fmt.status != "approved" else fmt.sugg
        fmt.example_file = latest["path"]
        fmt.last_seen = timezone.make_aware(f["last_seen"])
        fmt.file_count = max(fmt.file_count, f["file_count"])
        fmt.save()
        state = "existing" if fmt.status == "approved" else "new"
    for folder, n in f["folders"].items():
        FormatFolder.objects.update_or_create(format=fmt, folder_path=folder,
    DiscoveryItem.objects.create(run=run, format=fmt, file_count=f["file_coun
                                 folders=dict(f["folders"]), state=state)
    counts[state] += 1
    log.section(f"File type '{fmt.display_name}' ({state})")
    log.detail(f"files={f['file_count']} · first_seen={f['first_seen']:%Y-%m-
               f"last_seen={f['last_seen']:%Y-%m-%d} · delimiter={f['delimite
               f"columns={len(f['headers'])} · signature={sig}")
    log.detail("headers: " + " | ".join(f["headers"]))
    for folder, n in f["folders"].items():
        log.detail(f"folder: {folder} ({n} files)")
    log.detail(f"latest file profiled: {latest['path']} ({latest['info']['row
    for fld, g in guess.items():
        log.detail(f"suggest {fld} -> '{g['header']}' [{g['conf']}] {g['reaso
                   + (f" · date format {g.get('date_format')}" if g.get("date
    missing = [x for x in ("account_number", "chargeoff_date") if x not in gu
    if missing:
        log.warn(f"  no suggestion for: {', '.join(missing)}")
```

```
            log.detail(f"overall suggestion confidence: {suggestion['overall']}"
                       + (" (Artiva checked)" if rates else " (Artiva not checked)"))

            if state != "existing" and Setting.get("auto_approve", False):
                _maybe_auto_approve(fmt, guess)
                if fmt.status == "approved":
                    log.info(f"  auto-approved '{fmt.display_name}'")

        status_counts = {}
        for f in result["files"]:
            status_counts[f["status"]] = status_counts.get(f["status"], 0) + 1
        run.stats = {"files": len(result["files"]), "formats": len(result["formats"])
                     "by_status": status_counts, **counts}
        log.summary(run.stats)
        run.status = "done"
        run.finished_at = timezone.now()
        run.save()
        return run.stats


def _top_candidates(guess, headers, columns, synonyms):
    """Columns to check against Artiva: the guess plus any column that looks like
    from .automap import profile_column
    profiles = [profile_column(c) for c in columns]
    acct = sorted(range(len(headers)), key=lambda i: -profiles[i]["unique"] * (1
    dates = [i for i, p in enumerate(profiles) if p["date"] > 0.8][:3]
    if "account_number" in guess and guess["account_number"]["col"] not in acct:
        acct.insert(0, guess["account_number"]["col"])
    return {"account_number": acct, "chargeoff_date": dates}


def _suggestion_json(guess, delimiter):
    return {
        "fields": {f: {"header": g["header"], "col": g["col"], "conf": g["conf"],
                       "reason": g["reason"], "evidence": g["evidence"]} for f, g
        "date_format": guess.get("chargeoff_date", {}).get("date_format"),
        "date_format_label": DATE_LABEL_BY_CODE.get(guess.get("chargeoff_date", {
        "delimiter": delimiter,
        "overall": overall_confidence(guess),
    }


def _folder_had_other_format(folders, fmt):
    return FormatFolder.objects.filter(folder_path__in=list(folders)).exclude(for
        format__status="approved").exists()
```

```
def _maybe_auto_approve(fmt, guess):
    if overall_confidence(guess) != "HIGH":
        return
    folders = list(fmt.folders.values_list("folder_path", flat=True))
    # Needs an existing client assignment for every folder (from another format i
    assigned = {}
    for folder in folders:
        fc = FormatClient.objects.filter(folder_path=folder, filename_regex="").e
        if not fc:
            return
        assigned[folder] = fc
    with transaction.atomic():
        v = _new_version(fmt, {"delimiter": fmt.delimiter, "date_formats": [guess
                       to_field_maps(guess), "auto", "system", fmt.suggestion)
        for folder, fc in assigned.items():
            FormatClient.objects.update_or_create(
                format=fmt, folder_path=folder, filename_regex="",
                defaults={"client": fc.client, "client_source": fc.client_source,
                          "created_by": "system"})
        fmt.status = "approved"
        fmt.active_version = v
        fmt.save()


def _new_version(fmt, parse_config, field_maps, approval, user, evidence=None):
    cfg = {"quotechar": '"', "encoding": "utf-8-sig", "header_row": 0, "skip_foot
    cfg.update(parse_config)
    last = fmt.versions.order_by("-version").first()
    return FormatVersion.objects.create(format=fmt, version=(last.version + 1) if
                                        parse_config=cfg, field_maps=field_maps,
                                        approval=approval, approved_by=user)


def masked_sample(fmt, rows=5):
    """Headers + a few masked rows from the newest file of this format. Never sto
    path = fmt.example_file
    if not path or not os.path.exists(path):
        return {"headers": fmt.headers, "rows": [], "parsed_dates": {}}
    info, _ = read_sample(path, rows, delimiter=fmt.delimiter or None)
    if not info:
        return {"headers": fmt.headers, "rows": [], "parsed_dates": {}}
    cols = info["columns"]
    data = [[mask(cols[c][r]) for c in range(len(cols))] for r in range(min(rows,
    return {"headers": info["headers"], "rows": data}
```

```
def date_preview(fmt, header, label, rows=5):
    """Live 'parsed as' preview for the wizard. Only the chosen date column is re

    Returns {"samples": [{"raw","parsed"}], "rate": 0..1 over SAMPLE_ROWS, "sugge
    """
    from .common import detect_date_format, parse_date
    info, _ = read_sample(fmt.example_file, SAMPLE_ROWS, delimiter=fmt.delimiter
    if not info or norm(header) not in [norm(h) for h in info["headers"]]:
        return {"samples": [], "rate": 0.0, "suggested": None}
    col = info["columns"][[norm(h) for h in info["headers"]].index(norm(header))]
    code = DATE_FORMATS.get(label)
    filled = [v for v in col if v.strip()]
    ok = sum(1 for v in filled if code and parse_date(v, [code]))
    best, _ = detect_date_format(filled)
    samples = []
    for v in filled[:rows]:
        d = parse_date(v, [code]) if code else None
        samples.append({"raw": v, "parsed": d.isoformat() if d else None})
    return {"samples": samples, "rate": ok / max(len(filled), 1),
            "suggested": DATE_LABEL_BY_CODE.get(best)}


@transaction.atomic
def save_format_setup(fmt, payload, user):
    """Save the wizard's Step 3 for one format.

    payload = {
      "account_number": "<header>", "chargeoff_date": "<header>", "date_format":
      "client_ref": "<header>" | None,
      "client_name": {"source": "assigned"} | {"source": "file_column", "header":
      "skip_footer_rows": 0,
      "folders": [{"folder_path": "...", "file_pattern": "" | "ATT_PRE_*", "clien
    }
    """
    headers = fmt.headers
    hn = {norm(h): h for h in headers}
    errors = []
    for f in ("account_number", "chargeoff_date"):
        if norm(payload.get(f) or "") not in hn:
            errors.append(f"Pick a column for {f.replace('_', ' ')}.")
    if payload.get("account_number") and payload.get("account_number") == payload
        errors.append("Account Number and Charge-off Date must be different colum
    if payload.get("date_format") not in DATE_FORMATS:
        errors.append("Pick a date format.")
    cn = payload.get("client_name") or {"source": "assigned"}
```

```
    for fo in payload.get("folders", []):
        if cn["source"] == "assigned" and not fo.get("client_id"):
            errors.append(f"Pick a client for {fo['folder_path']}.")
    if errors:
        raise ValueError(errors)

    field_maps = {
        "account_number": {"source": "header", "header": hn[norm(payload["account
                           "index_fallback": headers.index(hn[norm(payload["accou
        "chargeoff_date": {"source": "header", "header": hn[norm(payload["chargeo
        "client_name": ({"source": "header", "header": cn["header"]} if cn["sourc
                        else {"source": "assigned_client"}),
    }
    if payload.get("client_ref"):
        field_maps["client_ref"] = {"source": "header", "header": payload["client

    v = _new_version(fmt, {"delimiter": fmt.delimiter, "skip_footer_rows": int(pa
                           "date_formats": [DATE_FORMATS[payload["date_format"]]]
                     field_maps, "human", user, fmt.suggestion)
    for fo in payload.get("folders", []):
        pattern = (fo.get("file_pattern") or "").strip()
        FormatClient.objects.update_or_create(
            format=fmt, folder_path=fo["folder_path"],
            filename_regex=wildcard_to_regex(pattern) if pattern else "",
            defaults={"file_pattern": pattern, "client_id": fo.get("client_id"),
                      "client_source": "file_column" if cn["source"] == "file_col
                      "source": "wizard", "created_by": user})
    fmt.status = "approved"
    fmt.active_version = v
    fmt.save()
    learn_synonyms(field_maps, user)
    return v


# ---------------------------------------------------------------- runtime proc

def client_for(fmt, path):
    """Most specific FormatClient: folder + regex, then folder only."""
    folder = os.path.dirname(path)
    name = os.path.basename(path)
    rows = list(FormatClient.objects.filter(format=fmt).select_related("client"))
    rows = [r for r in rows if norm(r.folder_path) == norm(folder)]
    for r in sorted(rows, key=lambda r: r.filename_regex == ""):
        if not r.filename_regex or re.match(r.filename_regex, name):
            return r
    return None
```

```
def _try_drift(path, headers):
    """Same folder, new layout: reuse a mapping if every mapped header still exis
    folder = os.path.dirname(path)
    hn = {norm(h) for h in headers}
    for ff in FormatFolder.objects.filter(folder_path=folder, format__status="app
            "format__active_version"):
        v = ff.format.active_version
        if not v:
            continue
        needed = [m["header"] for m in v.field_maps.values() if m.get("source") =
        if all(norm(h) in hn for h in needed):
            return ff.format, v
    return None, None


def process_new_files(run_id):
    run = Run.objects.get(pk=run_id)
    with run_logged(run) as log:
        return _process_new_files(run, log)


def _process_new_files(run, log):
    run_id = run.id
    log.info(f"Watched folders: {', '.join((run.params or {}).get('roots') or roo
    stats = {"seen": 0, "skipped_known": 0, "processed": 0, "partial": 0, "needs_
             "failed": 0, "headerless": 0, "accounts": 0}
    known = dict(PlacementFile.objects.values_list("file_hash", "id"))
    on_err = lambda p, m: log.error(f"[FOLDER] {p} - {m}")  # noqa: E731
    for root, path in iter_files((run.params or {}).get("roots") or roots(), on_e
        if os.path.splitext(path)[1].lower() not in FLAT_EXTS:
            stats["not_flat_file"] = stats.get("not_flat_file", 0) + 1
            log.detail(f"[SKIP] {path} - not a flat file type ({os.path.splitext(
            continue
        stats["seen"] += 1
        try:
            h = sha256_file(path)
        except OSError as exc:
            stats["failed"] += 1
            log.error(f"[UNREADABLE] {path} - {exc}")
            continue
        if h in known:
            stats["skipped_known"] += 1
            log.detail(f"[ALREADY DONE] {path} - same content as file #{known[h]}
            continue
```

```
            pf = process_file(path, run, file_hash=h, log=log)
            known[h] = pf.id
            stats[pf.status] = stats.get(pf.status, 0) + 1
            stats["accounts"] += pf.rows_ok
            Run.objects.filter(pk=run_id).update(progress=stats)
        run.stats = stats
        log.info(f"{stats['skipped_known']} files skipped (already processed earlier)
        run.export_path = export_run(run) if stats["accounts"] else ""
        log.info(f"Extract written: {run.export_path}" if run.export_path else "No ac
        log.summary(stats)
        removed = prune_logs(log_dir(), Setting.get("log_retention_days", 90))
        if removed:
            log.info(f"Deleted {removed} log files older than {Setting.get('log_reten
        run.status = "done"
        run.finished_at = timezone.now()
        run.save()
        return stats


    class _NullLog:
        def __getattr__(self, _):
            return lambda *a, **k: None


    def process_file(path, run, file_hash=None, existing=None, log=None):
        """Parse one file into pi_accounts with detailed logging. Returns the Placeme
        import time
        log = log or _NullLog()
        t0 = time.monotonic()
        log.section(os.path.basename(path))
        try:
            st = os.stat(path)
            log.detail(f"path={path}")
            log.detail(f"size={st.st_size:,} bytes · modified={datetime.fromtimestamp
                       f" · sha256={(file_hash or '')[:12] or 'n/a'}")
        except OSError:
            pass
        pf = _process_file(path, run, file_hash, existing, log)
        secs = time.monotonic() - t0
        rate = f" · {int(pf.rows_total / secs):,} rows/s" if pf.rows_total and secs >
        log.detail(f"took {secs:.1f}s{rate}")
        log_file_result(log if not isinstance(log, _NullLog) else None, pf)
        return pf


    def _process_file(path, run, file_hash=None, existing=None, log=None):
```

```
prod, segment = segment_for(path)
pf = existing or PlacementFile(
    run=run, file_name=os.path.basename(path), file_path=path,
    file_hash=file_hash or sha256_file(path), file_size=os.path.getsize(path)
    file_mtime=timezone.make_aware(datetime.fromtimestamp(os.path.getmtime(pa
    prod=prod, segment=segment)
pf.run = run

log.detail(f"prod={prod or '?'} · segment={segment or '?'}" + ("" if prod els
info, status = read_sample(path, 50)
if not info:
    pf.status = "headerless" if status in ("headerless", "no_delimiter_possib
    pf.message = {"headerless": "No header row (fixed width / multi-record) -
                  "no_delimiter_possible_fixed_width": "No delimiter found (f
                  "empty": "File is empty"}.get(status, f"Could not read file
    pf.save()
    return pf
pf.signature = signature(info["headers"])
log.detail(f"layout: delimiter={info['delimiter']!r} · columns={len(info['hea
log.detail("headers: " + " | ".join(info["headers"]))
fmt = Format.objects.filter(signature=pf.signature).select_related("active_ve

if fmt is None or fmt.status != "approved":
    log.detail("file type: " + (f"'{fmt.display_name}' is not set up yet ({fm
    d_fmt, d_ver = _try_drift(path, info["headers"])
    if d_fmt:
        fmt = _clone_for_drift(d_fmt, d_ver, info, path)
        log.warn(f"  LAYOUT CHANGED: reused mapping of '{d_fmt.display_name}'
                 f"(all mapped columns still present) -> new file type '{fmt.
    else:
        if fmt is None:
            Format.objects.create(signature=pf.signature, headers=info["heade
                                  status="needs_review", example_file=path, f
                                  display_name=_display_name({os.path.dirname
                                  suggestion=_suggestion_json(guess_mapping(i
                                                                              g
        pf.status = "needs_review"
        pf.message = "New file layout. Set it up in Set Up Formats or the Rev
        pf.save()
        return pf

v = fmt.active_version
log.detail(f"file type: '{fmt.display_name}' v{v.version} (set up via {v.appr
           + (f" by {v.approved_by}" if v.approved_by else "") + f" on {v.app
log.detail("columns: " + " · ".join(
    f"{k}={m.get('header') or m.get('source')}" for k, m in v.field_maps.item
```

```
    log.detail(f"config: date_formats={v.parse_config.get('date_formats')} · "
               f"skip_footer_rows={v.parse_config.get('skip_footer_rows', 0)} · e
    fc = client_for(fmt, path)
    if fc is None:
        pf.status = "needs_review"
        pf.message = "No client assigned for this folder / file name."
        pf.save()
        return pf
    pf.client = fc.client
    pf.format_version = fmt.active_version
    log.detail(f"client: {fc.client.client_code + ' - ' + fc.client.client_name i
               f" (matched {'pattern ' + fc.file_pattern if fc.filename_regex els

    was_saved = pf.pk is not None
    try:
        with transaction.atomic():
            _load_rows(pf, path, v, fc, log)
    except MappingError as exc:
        # Everything inside the atomic block was rolled back; record the failure
        if not was_saved:
            pf.pk = None
        pf.status, pf.message = "failed", str(exc)
        pf.rows_ok = pf.rows_error = pf.rows_total = 0
        pf.processed_at = timezone.now()
        pf.save()
    return pf


def _load_rows(pf, path, v, fc, log=None):
    pf.rows_ok = 0
    pf.save()
    pf.accounts.all().delete()
    pf.row_errors.all().delete()
    log = log or _NullLog()
    ok_rows, err_rows, total, bad = [], [], 0, set()
    first_rows = {}            # error_code -> first 10 account serial numbers (n
    next_progress = 50000
    records = iter_records(path, v.parse_config, v.field_maps,
                           fc.client.client_name if fc.client else None)
    for item in validate(records, v.parse_config.get("date_formats")):
        if item[0] == "ok":
            _, rn, rec = item
            ok_rows.append(Account(file=pf, row_num=rn, **rec))
            total += 1
        else:
            _, rn, field, code, seq = item
```

```
            err_rows.append(RowError(file=pf, row_num=rn, account_seq=seq, field=
            lst = first_rows.setdefault(code, [])
            if len(lst) < 10:
                lst.append(f"#{seq} (line {rn})")
            if rn not in bad:
                bad.add(rn)
                total += 1
        if len(ok_rows) >= INSERT_BATCH:
            Account.objects.bulk_create(ok_rows, batch_size=INSERT_BATCH)
            pf.rows_ok += len(ok_rows)
            ok_rows = []
            if pf.rows_ok >= next_progress:
                log.detail(f"... {pf.rows_ok:,} accounts loaded so far")
                next_progress += 50000
        if len(err_rows) >= INSERT_BATCH:
            RowError.objects.bulk_create(err_rows, batch_size=INSERT_BATCH)
            err_rows = []
    Account.objects.bulk_create(ok_rows, batch_size=INSERT_BATCH)
    RowError.objects.bulk_create(err_rows, batch_size=INSERT_BATCH)
    pf.rows_ok += len(ok_rows)
    pf.rows_total = total
    pf.rows_error = len(bad)
    pf.status = "partial" if bad else "processed"
    pf.message = ""
    pf.processed_at = timezone.now()
    pf.save()
    log.detail(f"records={total:,} · loaded={pf.rows_ok:,} · rejected={len(bad):,
    for code, rows in sorted(first_rows.items()):
        log.detail(f"rejected {code}: first at account " + ", ".join(rows)
                   + (" ..." if len(rows) == 10 else ""))


def _clone_for_drift(src_fmt, src_ver, info, path):
    fmt, _ = Format.objects.get_or_create(
        signature=signature(info["headers"]),
        defaults={"headers": info["headers"], "delimiter": info["delimiter"], "st
                  "example_file": path, "display_name": src_fmt.display_name + "
                  "notes": f"Auto-mapped from format #{src_fmt.id} after layout c
    cfg = dict(src_ver.parse_config, delimiter=info["delimiter"])
    v = _new_version(fmt, cfg, src_ver.field_maps, "auto_drift", "system")
    for fc in src_fmt.clients.all():
        FormatClient.objects.get_or_create(
            format=fmt, folder_path=fc.folder_path, filename_regex=fc.filename_re
            defaults={"file_pattern": fc.file_pattern, "client": fc.client, "clie
                      "source": fc.source, "created_by": "system"})
    FormatFolder.objects.get_or_create(format=fmt, folder_path=os.path.dirname(pa
```

```
    fmt.status, fmt.active_version = "approved", v
    fmt.save()
    return fmt


def reprocess_parked(fmt, user="system"):
    """After a file type is set up, process its waiting files (own run + log)."""
    run = Run.objects.create(mode="reprocess", triggered_by=user, params={"format
    with run_logged(run) as log:
        log.info(f"Processing files waiting for file type '{fmt.display_name}'")
        stats = {"files": 0, "accounts": 0}
        for pf in PlacementFile.objects.filter(signature=fmt.signature, status="n
            if not os.path.exists(pf.file_path):
                log.warn(f"[MISSING] {pf.file_path} - file no longer in its folde
                continue
            pf = process_file(pf.file_path, run, existing=pf, log=log)
            stats["files"] += 1
            stats["accounts"] += pf.rows_ok
            stats[pf.status] = stats.get(pf.status, 0) + 1
        run.stats = stats
        run.export_path = export_run(run) if stats["accounts"] else ""
        if run.export_path:
            log.info(f"Extract written: {run.export_path}")
        log.summary(stats)
        run.status, run.finished_at = "done", timezone.now()
        run.save()
    return run


def reprocess_file(pf, user):
    """Admin 'Reprocess' on one file (own run + log)."""
    run = Run.objects.create(mode="reprocess", triggered_by=user, params={"file_i
    with run_logged(run, user) as log:
        log.info(f"Manual reprocess of {pf.file_path}")
        pf = process_file(pf.file_path, run, existing=pf, log=log)
        run.stats = {"files": 1, "accounts": pf.rows_ok, pf.status: 1}
        run.export_path = export_run(run) if pf.rows_ok else ""
        log.summary(run.stats)
        run.status, run.finished_at = "done", timezone.now()
        run.save()
    return pf, run


# ---------------------------------------------------------------- export

EXPORT_COLS = ["file_name", "file_path", "account_seq", "line_number", "account_n
```

```
                "client_name", "client_ref"]


def export_rows(file_qs):
    for acc in Account.objects.filter(file__in=file_qs).select_related("file").or
            chunk_size=10000):
        yield [acc.file.file_name, acc.file.file_path, acc.account_seq, acc.row_n
               acc.chargeoff_date.isoformat() if acc.chargeoff_date else "",
               acc.client_name or "", acc.client_ref or ""]


def export_run(run):
    export_dir = Setting.get("export_dir")
    if not export_dir:
        return ""
    fmt = Setting.get("export_format", "csv")
    delim = "|" if fmt == "txt" else ","
    os.makedirs(export_dir, exist_ok=True)
    path = os.path.join(export_dir, f"placement_extract_run{run.id}_{timezone.now
    n = 0
    with open(path, "w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh, delimiter=delim)
        w.writerow(EXPORT_COLS)
        for row in export_rows(PlacementFile.objects.filter(run=run)):
            w.writerow(row)
            n += 1
    run.stats = dict(run.stats or {}, exported_rows=n)
    return path


# ------------------------------------------------------------------ Excel templa

def template_context():
    clients = {c.client_code: {"id": c.id, "client_name": c.client_name, "prod":
               for c in Client.objects.filter(active=True)}
    existing = {(norm(fc.folder_path), fc.filename_regex): fc.format_id for fc in
    return {
        "prods": set(SegmentDefault.objects.values_list("prod", flat=True)) | {c[
        "segments": set(SegmentDefault.objects.values_list("segment", flat=True))
        "roots": roots(), "clients": clients, "existing": existing, "sample_rows"
    }


def build_template(run_id=None):
    ctx = template_context()
    clients = [{"client_code": k, **v} for k, v in ctx["clients"].items()]
```

```
    prefill = None
    if run_id:
        items = DiscoveryItem.objects.filter(run_id=run_id).exclude(state="existi
        fmts = []
        for it in items:
            s = it.format.suggestion or {}
            guess = {f: {"header": g["header"]} for f, g in (s.get("fields") or {
            if "chargeoff_date" in guess:
                guess["chargeoff_date"]["date_format"] = s.get("date_format")
            fmts.append({"signature": it.format.signature, "headers": it.format.h
                         "delimiter": it.format.delimiter, "folders": it.folders,
                         "example_file": it.format.example_file})
        sd = {norm(s.folder_path): {"prod": s.prod, "segment": s.segment} for s i
        prefill = template_io.prefill_rows_from_formats(fmts, sd)
    return template_io.build_template(sorted(ctx["prods"]), sorted(ctx["segments"


def _result_json(r):
    return {"row_id": r.row_id, "values": r.values, "status": r.status, "messages
            "fixes": r.fixes, "resolved": r.resolved}


def template_dry_run(file_name, data, user):
    rows = template_io.read_template(data)      # raises ValueError with a user m
    ctx = template_context()
    results = [_result_json(template_io.validate_row(v, ctx)) for v in rows]
    return TemplateImport.objects.create(file_name=file_name, original=data, rows


def template_fix(imp, row_id, field, value):
    ctx = template_context()
    rows = imp.rows
    for i, r in enumerate(rows):
        if r["row_id"] == row_id:
            rows[i] = _result_json(template_io.validate_row(template_io.apply_fix
    imp.rows = rows
    imp.save(update_fields=["rows"])
    return imp


def template_errors_xlsx(imp):
    results = [template_io.RowResult(row_id=r["row_id"], values=r["values"], stat
                                     messages=r["messages"]) for r in imp.rows]
    return template_io.errors_workbook(bytes(imp.original), results)
```

```
@transaction.atomic
def template_commit(imp, user):
    saved, formats_touched = 0, set()
    for r in imp.rows:
        if r["status"] not in ("ready", "update"):
            continue
        res = r["resolved"]
        fmt, created = Format.objects.get_or_create(
            signature=res["signature"],
            defaults={"headers": res["headers"], "delimiter": res["parse_config"]
                      "status": "approved", "example_file": res["example_file"],
                      "display_name": _display_name({res["folder_path"]: 1}, res[
                      "notes": res.get("notes", "")})
        v = _new_version(fmt, res["parse_config"], res["field_maps"], "excel_temp
        FormatFolder.objects.update_or_create(format=fmt, folder_path=res["folder
                                              defaults={"file_count": res["matchi
        FormatClient.objects.update_or_create(
            format=fmt, folder_path=res["folder_path"], filename_regex=res["filen
            defaults={"file_pattern": r["values"]["file_pattern"], "client_id": r
                      "client_source": res["client_source"], "source": "excel_tem
        SegmentDefault.objects.get_or_create(folder_path=res["folder_path"],
                                             defaults={"prod": res["prod"], "segm
        fmt.status, fmt.active_version = "approved", v
        fmt.save()
        learn_synonyms(res["field_maps"], user)
        saved += 1
        formats_touched.add(fmt.id)
    imp.status = "committed"
    imp.save(update_fields=["status"])
    return {"saved": saved, "formats": sorted(formats_touched)}
```

B.12 `placement_intake/tasks.py`

> 🟡 **ADAPT · NOT RUN — register with iResolve's Celery app**

```
"""Celery tasks. Register a beat schedule for process_new_files_task (see setting
from celery import shared_task

from . import services
from .models import Format, Run


@shared_task(name="placement_intake.discover")
def discover_task(run_id):
    try:
```

```
        return services.run_discovery(run_id)
    except Exception as exc:  # noqa: BLE001 - surface the error on the run row f
        Run.objects.filter(pk=run_id).update(status="failed", error=str(exc)[:200
        raise


@shared_task(name="placement_intake.process")
def process_task(run_id):
    try:
        return services.process_new_files(run_id)
    except Exception as exc:  # noqa: BLE001
        Run.objects.filter(pk=run_id).update(status="failed", error=str(exc)[:200
        raise


@shared_task(name="placement_intake.process_scheduled")
def process_new_files_task():
    run = Run.objects.create(mode="process", triggered_by="schedule")
    return process_task(run.id)


@shared_task(name="placement_intake.reprocess_format")
def reprocess_format_task(format_id):
    return services.reprocess_parked(Format.objects.get(pk=format_id)).id
```

B.13 `placement_intake/permissions.py`

> 🟡 **ADAPT · NOT RUN — wire user_has_perm() to iResolve permissions**

```
"""Server-side permission checks. Hidden buttons in the UI are NOT security.

CODEX: wire user_has_perm() to iResolve's existing module-permission system
(the per-user module toggles in Admin Settings).
"""
from rest_framework.permissions import BasePermission

PERM_VIEW = "placement_intake.view"     # Overview, Files, download exports
PERM_SETUP = "placement_intake.setup"   # run discovery, set up formats, template
PERM_ADMIN = "placement_intake.admin"   # settings, clients, retire formats, repr


def user_has_perm(user, perm):
    # Replace with the iResolve module permission lookup.
    return user.is_authenticated and (user.is_superuser or user.has_perm(perm))
```

```
def _perm_class(perm):
    class _P(BasePermission):
        message = "You do not have access to this part of Placement Intake."

        def has_permission(self, request, view):
            return user_has_perm(request.user, perm)
    _P.__name__ = f"Has_{perm.replace('.', '_')}"
    return _P


CanView = _perm_class(PERM_VIEW)
CanSetup = _perm_class(PERM_SETUP)
CanAdmin = _perm_class(PERM_ADMIN)
```

B.14 `placement_intake/views.py`

> 🟡 **ADAPT · NOT RUN — assumes DRF**

```
"""DRF API for Placement Intake. All responses are JSON unless noted (xlsx/csv do
import csv
import os
import re
from datetime import timedelta

from django.db.models import Count, Q, Sum
from django.http import FileResponse, HttpResponse, StreamingHttpResponse
from django.shortcuts import get_object_or_404
from django.utils import timezone
from rest_framework.decorators import api_view, parser_classes, permission_classe
from rest_framework.parsers import JSONParser, MultiPartParser
from rest_framework.response import Response

from . import services
from .common import DATE_FORMATS, DELIMITER_LABEL, FIELD_LABELS, norm, wildcard_t
from .models import (Client, DiscoveryFile, DiscoveryItem, Format, FormatClient,
                     PlacementFile, Run, SegmentDefault, Setting, TemplateImport)
from .permissions import CanAdmin, CanSetup, CanView
from .tasks import discover_task, process_task, reprocess_format_task

XLSX = "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"


def _user(request):
    return getattr(request.user, "username", "") or "unknown"
```

```
def _bad(errors, code=400):
    return Response({"errors": errors if isinstance(errors, list) else [str(error


# ---------------------------------------------------------------- overview

@api_view(["GET"])
@permission_classes([CanView])
def overview(request):
    since = timezone.now() - timedelta(days=7)
    files = PlacementFile.objects.filter(processed_at__gte=since)
    last = Run.objects.filter(mode="process", status="done").first()
    by_prod = (PlacementFile.objects.filter(processed_at__date=timezone.localdate
               .values("prod").annotate(files=Count("id"), accounts=Sum("rows_ok"
               .order_by("prod"))
    return Response({
        "processed_7d": files.filter(status__in=["processed", "partial"]).count()
        "formats_need_review": Format.objects.filter(status="needs_review").count
        "files_waiting": PlacementFile.objects.filter(status="needs_review").coun
        "files_failed": PlacementFile.objects.filter(status="failed").count(),
        "layout_changes_7d": Format.objects.filter(versions__approval="auto_drift
                                                   versions__approved_at__gte=sin
        "last_run": last and {"id": last.id, "finished_at": last.finished_at, "st
                              "has_export": bool(last.export_path), "has_log": bo
        "running": [{"id": r.id, "mode": r.mode} for r in Run.objects.filter(stat
        "today_by_prod": list(by_prod),
    })


# ---------------------------------------------------------------- runs

@api_view(["POST"])
@permission_classes([CanSetup])
def start_run(request):
    mode = request.data.get("mode")
    if mode not in ("discovery", "process"):
        return _bad("mode must be discovery or process")
    if Run.objects.filter(mode=mode, status__in=["queued", "running"]).exists():
        return _bad("A run of this type is already in progress. Wait for it to fi
    roots = request.data.get("roots") or services.roots()
    allowed = [norm(r) for r in services.roots()]
    if any(not any(norm(r).startswith(a) for a in allowed) for r in roots):
        return _bad("One or more folders are outside the watched placement folder
    run = Run.objects.create(mode=mode, triggered_by=_user(request), params={
```

```
        "roots": roots, "since_days": request.data.get("since_days"),
        "sample_rows": int(request.data.get("sample_rows") or services.SAMPLE_ROW
    (discover_task if mode == "discovery" else process_task).delay(run.id)
    return Response({"run_id": run.id}, status=202)


@api_view(["GET"])
@permission_classes([CanView])
def run_detail(request, run_id):
    r = get_object_or_404(Run, pk=run_id)
    return Response({"id": r.id, "mode": r.mode, "status": r.status, "progress":
                     "stats": r.stats, "error": r.error, "started_at": r.started_
                     "finished_at": r.finished_at, "has_export": bool(r.export_pa
                     "has_log": bool(r.log_path and os.path.exists(r.log_path))})


@api_view(["GET"])
@permission_classes([CanView])
def run_formats(request, run_id):
    """Wizard step 2: formats in this discovery run + parked/skipped files."""
    items = DiscoveryItem.objects.filter(run_id=run_id).select_related("format").
    out = []
    for it in items:
        f = it.format
        s = f.suggestion or {}
        fields = s.get("fields", {})
        out.append({
            "format_id": f.id, "display_name": f.display_name, "state": it.state,
            "file_count": it.file_count, "folders": it.folders, "column_count": l
            "delimiter": DELIMITER_LABEL.get(f.delimiter, f.delimiter),
            "prefilled": sum(1 for k in ("account_number", "chargeoff_date", "cli
            "confidence": s.get("overall", "LOW"),
        })
    parked = (DiscoveryFile.objects.filter(run_id=run_id).exclude(status="ok")
              .values("file_path", "folder", "status").order_by("status", "file_p
    return Response({"formats": out, "parked": list(parked)})


@api_view(["GET"])
@permission_classes([CanView])
def run_export(request, run_id):
    run = get_object_or_404(Run, pk=run_id)
    return _csv_response(services.export_rows(PlacementFile.objects.filter(run=ru
                         f"placement_extract_run{run.id}.csv")
```

```
def _csv_response(rows, filename):
    class Echo:
        def write(self, v):
            return v
    w = csv.writer(Echo())

    def gen():
        yield w.writerow(services.EXPORT_COLS)
        for r in rows:
            yield w.writerow(r)
    resp = StreamingHttpResponse(gen(), content_type="text/csv")
    resp["Content-Disposition"] = f'attachment; filename="{filename}"'
    return resp


@api_view(["GET"])
@permission_classes([CanView])
def run_log_download(request, run_id):
    """Download the run's .log file. Works while the run is still going (file is
    run = get_object_or_404(Run, pk=run_id)
    if not run.log_path or not os.path.exists(run.log_path):
        return _bad("No log file for this run (it may have been deleted after the
    resp = FileResponse(open(run.log_path, "rb"), content_type="text/plain; chars
    resp["Content-Disposition"] = f'attachment; filename="{os.path.basename(run.l
    return resp


@api_view(["GET"])
@permission_classes([CanView])
def run_log_tail(request, run_id):
    """Last N lines for the in-app log viewer."""
    from .run_log import tail
    run = get_object_or_404(Run, pk=run_id)
    n = min(int(request.query_params.get("lines", 500)), 2000)
    return Response({"lines": tail(run.log_path, n), "status": run.status})


@api_view(["GET"])
@permission_classes([CanView])
def run_list(request):
    qs = Run.objects.all()
    if request.query_params.get("mode"):
        qs = qs.filter(mode=request.query_params["mode"])
    return Response({"runs": [{"id": r.id, "mode": r.mode, "status": r.status, "t
                               "started_at": r.started_at, "finished_at": r.finis
                               "has_export": bool(r.export_path), "has_log": bool
```

```
                                for r in qs[:100]]})


# ------------------------------------------------------------------ folders (wizar

@api_view(["GET"])
@permission_classes([CanSetup])
def folder_tree(request):
    """Children of a folder (or the roots). Only folders, never file contents."""
    path = request.query_params.get("path")
    allowed = services.roots()
    if not path:
        nodes = [{"path": r, "name": r, "has_children": True} for r in allowed]
    else:
        if not any(norm(path).startswith(norm(r)) for r in allowed):
            return _bad("Folder is outside the watched placement folders.", 403)
        try:
            entries = sorted(os.scandir(path), key=lambda e: e.name.lower())
        except OSError:
            return _bad("Cannot open this folder.", 404)
        nodes = []
        for e in entries:
            if e.is_dir():
                try:
                    has_children = any(x.is_dir() for x in os.scandir(e.path))
                    file_count = sum(1 for x in os.scandir(e.path) if x.is_file()
                except OSError:
                    has_children, file_count = False, 0
                nodes.append({"path": e.path, "name": e.name, "has_children": has
                              "file_count": file_count})
    for n in nodes:
        n["prod"], n["segment"] = services.segment_for(n["path"])
    return Response({"nodes": nodes})


# ------------------------------------------------------------------ formats

def _format_summary(f):
    return {"id": f.id, "display_name": f.display_name, "status": f.status, "file
            "folders": list(f.folders.values_list("folder_path", flat=True)),
            "last_seen": f.last_seen, "version": f.active_version.version if f.ac
            "confidence": (f.suggestion or {}).get("overall")}


@api_view(["GET"])
@permission_classes([CanView])
```

```
def format_list(request):
    qs = Format.objects.select_related("active_version").prefetch_related("folder
    if request.query_params.get("status"):
        qs = qs.filter(status=request.query_params["status"])
    if request.query_params.get("filter") == "changed":
        qs = qs.filter(versions__approval="auto_drift").distinct()
    q = request.query_params.get("q")
    if q:
        qs = qs.filter(Q(display_name__icontains=q) | Q(folders__folder_path__ico
                       Q(headers__icontains=q)).distinct()
    order = "-file_count" if request.query_params.get("sort") != "recent" else "-
    return Response({"formats": [_format_summary(f) for f in qs.order_by(order)[:


@api_view(["GET"])
@permission_classes([CanView])
def format_detail(request, format_id):
    f = get_object_or_404(Format.objects.select_related("active_version"), pk=for
    s = f.suggestion or {}
    v = f.active_version
    current = None
    if v:
        current = {"field_maps": v.field_maps,
                   "date_format": next((k for k, c in DATE_FORMATS.items()
                                        if c in v.parse_config.get("date_formats"
                   "skip_footer_rows": v.parse_config.get("skip_footer_rows", 0)}
    waiting = PlacementFile.objects.filter(signature=f.signature, status="needs_r
    return Response({
        **_format_summary(f),
        "headers": f.headers, "delimiter": DELIMITER_LABEL.get(f.delimiter, f.del
        "suggestion": s, "current": current, "notes": f.notes, "files_waiting": w
        "field_labels": FIELD_LABELS, "date_formats": list(DATE_FORMATS),
        "folder_counts": {ff.folder_path: ff.file_count for ff in f.folders.all()
        "clients": [{"folder_path": c.folder_path, "file_pattern": c.file_pattern
                     "client_id": c.client_id, "client_label": c.client and f"{c.
                     "client_source": c.client_source} for c in f.clients.select_
        "versions": [{"version": x.version, "approval": x.approval, "approved_by"
                      "approved_at": x.approved_at, "field_maps": x.field_maps}
                     for x in f.versions.order_by("-version")],
    })


@api_view(["GET"])
@permission_classes([CanSetup])
def format_sample(request, format_id):
    return Response(services.masked_sample(get_object_or_404(Format, pk=format_id
```

```
@api_view(["GET"])
@permission_classes([CanSetup])
def format_date_preview(request, format_id):
    f = get_object_or_404(Format, pk=format_id)
    return Response(services.date_preview(f, request.query_params.get("header", "
                                          request.query_params.get("date_format",


@api_view(["POST"])
@permission_classes([CanSetup])
def format_pattern_test(request, format_id):
    """How many files in a folder match a wildcard pattern (live count in the spl
    folder, pattern = request.data.get("folder_path", ""), request.data.get("file
    if not any(norm(folder).startswith(norm(r)) for r in services.roots()):
        return _bad("Folder is outside the watched placement folders.", 403)
    try:
        rx = re.compile(wildcard_to_regex(pattern))
        names = [n for n in os.listdir(folder) if os.path.isfile(os.path.join(fol
    except (OSError, re.error):
        return _bad("Cannot read this folder or pattern.")
    matches = [n for n in names if rx.match(n)]
    return Response({"count": len(matches), "examples": matches[:3]})


@api_view(["POST"])
@permission_classes([CanSetup])
def format_setup(request, format_id):
    f = get_object_or_404(Format, pk=format_id)
    try:
        v = services.save_format_setup(f, request.data, _user(request))
    except ValueError as exc:
        return _bad(exc.args[0])
    waiting = PlacementFile.objects.filter(signature=f.signature, status="needs_r
    if waiting:
        reprocess_format_task.delay(f.id)
    return Response({"version": v.version, "files_queued": waiting})


@api_view(["POST"])
@permission_classes([CanAdmin])
def format_retire(request, format_id):
    f = get_object_or_404(Format, pk=format_id)
    f.status = "retired"
    f.save(update_fields=["status"])
```

```
    return Response({"status": "retired"})


# ---------------------------------------------------------------- review queue

@api_view(["GET"])
@permission_classes([CanView])
def review_queue(request):
    fmts = Format.objects.filter(status="needs_review").prefetch_related("folders
    out = []
    for f in fmts:
        fields = (f.suggestion or {}).get("fields", {})
        need = sum(1 for k in ("account_number", "chargeoff_date") if fields.get(
        out.append({**_format_summary(f), "fields_to_check": need + 1,  # +1: cli
                    "files_waiting": PlacementFile.objects.filter(signature=f.sig
                                                                  status="needs_r
    no_client = (PlacementFile.objects.filter(status="needs_review", message__sta
                 .values("signature").annotate(n=Count("id")))
    return Response({"formats": out, "files_missing_client": list(no_client)})


# ---------------------------------------------------------------- files

@api_view(["GET"])
@permission_classes([CanView])
def file_list(request):
    qs = PlacementFile.objects.select_related("client")
    p = request.query_params
    if p.get("status"):
        qs = qs.filter(status__in=p["status"].split(","))
    if p.get("prod"):
        qs = qs.filter(prod=p["prod"])
    if p.get("q"):
        qs = qs.filter(Q(file_name__icontains=p["q"]) | Q(file_path__icontains=p[
    if p.get("from"):
        qs = qs.filter(processed_at__date__gte=p["from"])
    if p.get("to"):
        qs = qs.filter(processed_at__date__lte=p["to"])
    page, size = int(p.get("page", 1)), min(int(p.get("page_size", 50)), 200)
    total = qs.count()
    rows = qs[(page - 1) * size: page * size]
    return Response({"total": total, "page": page, "page_size": size, "files": [{
        "id": f.id, "run_id": f.run_id, "file_name": f.file_name, "folder": os.pa
        "client": f.client and f.client.client_name, "status": f.status, "rows_ok
        "rows_error": f.rows_error, "processed_at": f.processed_at, "message": f.
        "signature": f.signature} for f in rows]})
```

```
@api_view(["GET"])
@permission_classes([CanView])
def file_errors(request, file_id):
    f = get_object_or_404(PlacementFile, pk=file_id)
    summary = f.row_errors.values("error_code").annotate(n=Count("id")).order_by(
    rows = f.row_errors.order_by("row_num").values("account_seq", "row_num", "fie
    return Response({"summary": list(summary), "rows": list(rows), "truncated": f


@api_view(["POST"])
@permission_classes([CanAdmin])
def file_reprocess(request, file_id):
    f = get_object_or_404(PlacementFile, pk=file_id)
    if not os.path.exists(f.file_path):
        return _bad("The file is no longer in its folder.")
    f, run = services.reprocess_file(f, _user(request))
    return Response({"status": f.status, "rows_ok": f.rows_ok, "rows_error": f.ro
                     "run_id": run.id, "has_log": bool(run.log_path)})


@api_view(["GET"])
@permission_classes([CanView])
def file_export(request, file_id):
    f = get_object_or_404(PlacementFile, pk=file_id)
    return _csv_response(services.export_rows(PlacementFile.objects.filter(pk=f.p
                         f"{os.path.splitext(f.file_name)[0]}_extract.csv")


# ---------------------------------------------------------------- clients & sett

@api_view(["GET"])
@permission_classes([CanView])
def client_list(request):
    qs = Client.objects.filter(active=True)
    if request.query_params.get("prod"):
        qs = qs.filter(prod=request.query_params["prod"])
    if request.query_params.get("q"):
        q = request.query_params["q"]
        qs = qs.filter(Q(client_code__icontains=q) | Q(client_name__icontains=q))
    return Response({"clients": [{"id": c.id, "code": c.client_code, "name": c.cl
                                  for c in qs[:200]]})


@api_view(["GET", "PUT"])
```

```
@permission_classes([CanAdmin])
def settings_view(request):
    keys = ["roots", "export_dir", "export_format", "auto_approve", "schedule_min
            "log_dir", "log_retention_days"]
    if request.method == "PUT":
        for k in keys:
            if k in request.data:
                Setting.objects.update_or_create(key=k, defaults={"value": reques
    return Response({k: Setting.get(k) for k in keys})


@api_view(["POST"])
@permission_classes([CanAdmin])
@parser_classes([MultiPartParser])
def import_reference_csv(request, kind):
    """CSV upload for clients (client_code,client_name,prod) or segment defaults
    (folder_path,prod,segment,date_format). Upserts; returns counts."""
    f = request.FILES.get("file")
    if not f:
        return _bad("Attach a CSV file.")
    rows = list(csv.DictReader(f.read().decode("utf-8-sig").splitlines()))
    n = 0
    for r in rows:
        if kind == "clients" and r.get("client_code"):
            Client.objects.update_or_create(client_code=r["client_code"].strip(),
                "client_name": r.get("client_name", "").strip(), "prod": r.get("p
            n += 1
        elif kind == "segments" and r.get("folder_path"):
            SegmentDefault.objects.update_or_create(folder_path=r["folder_path"].
                "prod": r.get("prod", "").strip(), "segment": r.get("segment", ""
                "date_format": r.get("date_format", "").strip()})
            n += 1
    return Response({"saved": n, "rows": len(rows)})


@api_view(["GET"])
@permission_classes([CanAdmin])
def segment_list(request):
    return Response({"segments": list(SegmentDefault.objects.values().order_by("f


@api_view(["GET", "DELETE"])
@permission_classes([CanAdmin])
def synonyms_view(request):
    if request.method == "DELETE":
        HeaderSynonym.objects.filter(field=request.query_params.get("field"),
```

```
                                        alias_norm=request.query_params.get("alias")
    return Response({"synonyms": list(HeaderSynonym.objects.values("field", "alia
                                        .order_by("field", "alias_norm"))})


# ---------------------------------------------------------------- Excel template

@api_view(["GET"])
@permission_classes([CanSetup])
def template_download(request):
    run_id = request.query_params.get("run_id")
    data = services.build_template(int(run_id) if run_id else None)
    name = f"placement_file_types_{'prefilled_run' + run_id if run_id else 'blank
    resp = HttpResponse(data, content_type=XLSX)
    resp["Content-Disposition"] = f'attachment; filename="{name}"'
    return resp


def _import_json(imp):
    counts = {"ready": 0, "update": 0, "error": 0}
    for r in imp.rows:
        counts[r["status"]] = counts.get(r["status"], 0) + 1
    rows = [{k: r[k] for k in ("row_id", "status", "messages", "fixes")} |
            {"folder_path": r["values"].get("folder_path"), "file_pattern": r["va
             "client": r["values"].get("client_code"),
             "matching_files": (r.get("resolved") or {}).get("matching_files")} f
    return {"id": imp.id, "file_name": imp.file_name, "status": imp.status, "coun


@api_view(["POST"])
@permission_classes([CanSetup])
@parser_classes([MultiPartParser])
def template_upload(request):
    f = request.FILES.get("file")
    if not f or not f.name.lower().endswith(".xlsx"):
        return _bad("Upload the .xlsx template file.")
    if f.size > 5 * 1024 * 1024:
        return _bad("The file is larger than 5 MB. Is this the right template?")
    try:
        imp = services.template_dry_run(f.name, f.read(), _user(request))
    except ValueError as exc:
        return _bad(str(exc))
    return Response(_import_json(imp))


@api_view(["GET"])
```

```
@permission_classes([CanSetup])
def template_import_detail(request, import_id):
    return Response(_import_json(get_object_or_404(TemplateImport, pk=import_id))


@api_view(["POST"])
@permission_classes([CanSetup])
@parser_classes([JSONParser])
def template_import_fix(request, import_id):
    imp = get_object_or_404(TemplateImport, pk=import_id, status="validated")
    services.template_fix(imp, int(request.data["row_id"]), request.data["field"]
    return Response(_import_json(imp))


@api_view(["POST"])
@permission_classes([CanSetup])
def template_import_commit(request, import_id):
    imp = get_object_or_404(TemplateImport, pk=import_id, status="validated")
    result = services.template_commit(imp, _user(request))
    for fid in result["formats"]:
        reprocess_format_task.delay(fid)
    return Response(result)


@api_view(["POST"])
@permission_classes([CanSetup])
def template_import_cancel(request, import_id):
    TemplateImport.objects.filter(pk=import_id, status="validated").update(status
    return Response({"status": "cancelled"})


@api_view(["GET"])
@permission_classes([CanSetup])
def template_import_errors(request, import_id):
    imp = get_object_or_404(TemplateImport, pk=import_id)
    resp = HttpResponse(services.template_errors_xlsx(imp), content_type=XLSX)
    resp["Content-Disposition"] = f'attachment; filename="errors_{imp.file_name}"
    return resp
```

B.15 `placement_intake/urls.py`

> 🟡 **ADAPT · NOT RUN**

```
"""Mount in the project urls.py:  path("api/placement-intake/", include("placemen
from django.urls import path
```

```
from . import views as v

urlpatterns = [
    path("overview/", v.overview),
    path("runs/", v.start_run),
    path("runs/list/", v.run_list),
    path("runs/<int:run_id>/", v.run_detail),
    path("runs/<int:run_id>/formats/", v.run_formats),
    path("runs/<int:run_id>/export.csv", v.run_export),
    path("runs/<int:run_id>/log.txt", v.run_log_download),
    path("runs/<int:run_id>/log/", v.run_log_tail),
    path("folders/", v.folder_tree),
    path("formats/", v.format_list),
    path("formats/<int:format_id>/", v.format_detail),
    path("formats/<int:format_id>/sample/", v.format_sample),
    path("formats/<int:format_id>/date-preview/", v.format_date_preview),
    path("formats/<int:format_id>/pattern-test/", v.format_pattern_test),
    path("formats/<int:format_id>/setup/", v.format_setup),
    path("formats/<int:format_id>/retire/", v.format_retire),
    path("review/", v.review_queue),
    path("files/", v.file_list),
    path("files/<int:file_id>/errors/", v.file_errors),
    path("files/<int:file_id>/reprocess/", v.file_reprocess),
    path("files/<int:file_id>/export.csv", v.file_export),
    path("clients/", v.client_list),
    path("settings/", v.settings_view),
    path("settings/segments/", v.segment_list),
    path("settings/synonyms/", v.synonyms_view),
    path("settings/import/<str:kind>/", v.import_reference_csv),
    path("template/", v.template_download),
    path("template/imports/", v.template_upload),
    path("template/imports/<int:import_id>/", v.template_import_detail),
    path("template/imports/<int:import_id>/fix/", v.template_import_fix),
    path("template/imports/<int:import_id>/commit/", v.template_import_commit),
    path("template/imports/<int:import_id>/cancel/", v.template_import_cancel),
    path("template/imports/<int:import_id>/errors.xlsx", v.template_import_errors
]
```

B.16 `placement_intake/management/commands/pi_seed.py`

> 🟡 **ADAPT · NOT RUN**

```
"""python manage.py pi_seed  -> seeds header synonyms and default settings (safe
from django.core.management.base import BaseCommand
```

```
from placement_intake.models import Setting
from placement_intake.services import seed_synonyms

DEFAULTS = {"roots": [], "export_dir": "", "export_format": "csv", "auto_approve"
            "schedule_minutes": 30, "log_dir": "", "log_retention_days": 90}


class Command(BaseCommand):
    help = "Seed Placement Intake synonyms and default settings"

    def handle(self, *args, **opts):
        seed_synonyms()
        for k, v in DEFAULTS.items():
            Setting.objects.get_or_create(key=k, defaults={"value": v})
        self.stdout.write(self.style.SUCCESS("Placement Intake seeded."))
```

B.17 `frontend/placement-intake/api.js`

> 🟠 **REFERENCE UI · NOT COMPILED — replace with iResolve HTTP helper**

```
// Thin API client. CODEX: replace fetch with iResolve's existing API helper (aut
const BASE = "/api/placement-intake";

async function req(method, path, body, isForm) {
  const opts = { method, credentials: "include", headers: {} };
  if (body !== undefined) {
    if (isForm) opts.body = body;
    else { opts.body = JSON.stringify(body); opts.headers["Content-Type"] = "appl
  }
  const res = await fetch(`${BASE}${path}`, opts);
  const data = res.headers.get("content-type")?.includes("json") ? await res.json
  if (!res.ok) {
    const err = new Error((data?.errors || [`Request failed (${res.status})`]).jo
    err.errors = data?.errors || [];
    err.status = res.status;
    throw err;
  }
  return data;
}

export const api = {
  overview: () => req("GET", "/overview/"),
  startRun: (mode, roots, since_days) => req("POST", "/runs/", { mode, roots, sin
  run: (id) => req("GET", `/runs/${id}/`),
```

```
  runFormats: (id) => req("GET", `/runs/${id}/formats/`),
  folders: (path) => req("GET", `/folders/${path ? `?path=${encodeURIComponent(pa
  formats: (params = {}) => req("GET", `/formats/?${new URLSearchParams(params)}`
  format: (id) => req("GET", `/formats/${id}/`),
  sample: (id) => req("GET", `/formats/${id}/sample/`),
  datePreview: (id, header, date_format) =>
    req("GET", `/formats/${id}/date-preview/?${new URLSearchParams({ header, date
  patternTest: (id, folder_path, file_pattern) =>
    req("POST", `/formats/${id}/pattern-test/`, { folder_path, file_pattern }),
  saveSetup: (id, payload) => req("POST", `/formats/${id}/setup/`, payload),
  retire: (id) => req("POST", `/formats/${id}/retire/`),
  review: () => req("GET", "/review/"),
  files: (params = {}) => req("GET", `/files/?${new URLSearchParams(params)}`),
  fileErrors: (id) => req("GET", `/files/${id}/errors/`),
  reprocess: (id) => req("POST", `/files/${id}/reprocess/`),
  clients: (prod, q) => req("GET", `/clients/?${new URLSearchParams({ prod: prod
  uploadTemplate: (file) => { const f = new FormData(); f.append("file", file); r
  importFix: (id, row_id, field, value) => req("POST", `/template/imports/${id}/f
  importCommit: (id) => req("POST", `/template/imports/${id}/commit/`),
  importCancel: (id) => req("POST", `/template/imports/${id}/cancel/`),
  // Downloads are plain links (window.location / <a href>) so the browser handle
  templateUrl: (runId) => `${BASE}/template/${runId ? `?run_id=${runId}` : ""}`,
  importErrorsUrl: (id) => `${BASE}/template/imports/${id}/errors.xlsx`,
  runExportUrl: (id) => `${BASE}/runs/${id}/export.csv`,
  runLogUrl: (id) => `${BASE}/runs/${id}/log.txt`,
  runLogTail: (id, lines = 500) => req("GET", `/runs/${id}/log/?lines=${lines}`),
  runs: (mode) => req("GET", `/runs/list/${mode ? `?mode=${mode}` : ""}`),
  fileExportUrl: (id) => `${BASE}/files/${id}/export.csv`,
};
```

B.18 `frontend/placement-intake/useRunPolling.js`

> 🟠 **REFERENCE UI · NOT COMPILED**

```
// Poll a run every 2s while it is queued/running; stop on done/failed or unmount
import { useEffect, useRef, useState } from "react";
import { api } from "./api";

export function useRunPolling(runId, { onDone } = {}) {
  const [run, setRun] = useState(null);
  const [error, setError] = useState(null);
  const doneRef = useRef(onDone);
  doneRef.current = onDone;

  useEffect(() => {
```

```
    if (!runId) return undefined;
    let timer;
    let alive = true;
    const tick = async () => {
      try {
        const r = await api.run(runId);
        if (!alive) return;
        setRun(r);
        if (r.status === "done" || r.status === "failed") {
          doneRef.current?.(r);
          return; // stop polling
        }
      } catch (e) {
        if (!alive) return;
        setError(e.message); // keep polling: a single network blip must not kill
      }
      timer = setTimeout(tick, 2000);
    };
    tick();
    return () => { alive = false; clearTimeout(timer); };
  }, [runId]);

  return { run, error };
}
```

B.19 `frontend/placement-intake/FileTypeSetup.jsx`

> 🟠 **REFERENCE UI · NOT COMPILED — keep state/validation/behaviour, restyle with iResolve components**

```
// Wizard Step 3 / Review panel: pick columns + assign clients for ONE file type.
// Logic is complete; markup uses plain elements + classNames. CODEX: swap in iRe
// components (Select, Button, Table, Chip, Toast) but KEEP the state, validation
import { useEffect, useMemo, useRef, useState } from "react";
import { api } from "./api";

const FIELDS = [
  { key: "account_number", label: "Account Number", required: true },
  { key: "chargeoff_date", label: "Charge-off Date", required: true },
  { key: "client_ref", label: "Client Reference", required: false },
];
const NOT_IN_FILE = "";

function initialState(detail) {
  const cur = detail.current?.field_maps;
```

```
  const sug = detail.suggestion?.fields || {};
  const pick = (k) => (cur?.[k]?.source === "header" ? cur[k].header : sug[k]?.he
  const clientFromFile = cur?.client_name?.source === "header";
  // One row per folder; existing split-by-pattern rows come back from detail.cli
  const rows = Object.entries(detail.folder_counts).map(([folder_path, files]) =>
    const existing = detail.clients.find((c) => c.folder_path === folder_path &&
    return { key: folder_path, folder_path, files, file_pattern: "", client_id: e
             client_label: existing?.client_label ?? null };
  });
  detail.clients.filter((c) => c.file_pattern).forEach((c) =>
    rows.push({ key: `${c.folder_path}|${c.file_pattern}`, folder_path: c.folder_
                file_pattern: c.file_pattern, client_id: c.client_id, client_labe
  return {
    columns: { account_number: pick("account_number"), chargeoff_date: pick("char
               client_ref: pick("client_ref") },
    dateFormat: detail.current?.date_format || detail.suggestion?.date_format_lab
    clientSource: clientFromFile ? "file_column" : "assigned",
    clientHeader: clientFromFile ? cur.client_name.header : "",
    skipFooterRows: detail.current?.skip_footer_rows ?? 0,
    folderRows: rows,
  };
}

export default function FileTypeSetup({ detail, sample, onSaved, onSkip, prod })
  const [state, setState] = useState(() => initialState(detail));
  const [activeField, setActiveField] = useState("account_number"); // target of
  const [datePreview, setDatePreview] = useState(null);
  const [saving, setSaving] = useState(false);
  const [serverErrors, setServerErrors] = useState([]);
  const touched = useRef(new Set());

  const set = (patch) => setState((s) => ({ ...s, ...patch }));
  const setColumn = (field, header) => {
    touched.current.add(field);
    setState((s) => {
      const columns = { ...s.columns };
      // A column can map to only one field: un-assign it elsewhere first.
      Object.keys(columns).forEach((k) => { if (k !== field && header && columns[
      columns[field] = header;
      return { ...s, columns };
    });
  };

  // Live date preview, debounced 300ms, whenever the date column or format chang
  useEffect(() => {
    const h = state.columns.chargeoff_date;
```

```
  if (!h || !state.dateFormat) { setDatePreview(null); return undefined; }
  const t = setTimeout(() => {
    api.datePreview(detail.id, h, state.dateFormat).then(setDatePreview).catch(
  }, 300);
  return () => clearTimeout(t);
}, [detail.id, state.columns.chargeoff_date, state.dateFormat]);

const problems = useMemo(() => {
  const p = [];
  if (!state.columns.account_number) p.push("Pick the Account Number column.");
  if (!state.columns.chargeoff_date) p.push("Pick the Charge-off Date column.")
  if (!state.dateFormat) p.push("Pick the date format.");
  if (datePreview && datePreview.rate < 0.95)
    p.push(`The date format reads only ${Math.round(datePreview.rate * 100)}% o
  if (state.clientSource === "file_column" && !state.clientHeader) p.push("Pick
  if (state.clientSource === "assigned" && state.folderRows.some((r) => !r.clie
    p.push("Pick a client for every folder.");
  if (state.folderRows.some((r) => r.files === null && !r.file_pattern.trim()))
    p.push("Fill in or remove the empty file name pattern.");
  return p;
}, [state, datePreview]);

// Click / number key: assign to the active field, then move to the next empty
const assignToActive = (header) => {
  const field = activeField;
  setColumn(field, header);
  const after = { ...state.columns, [field]: header };
  const next = FIELDS.find((f) => f.key !== field && !after[f.key] && f.require
            || FIELDS.find((f) => f.key !== field && !after[f.key]);
  if (next) setActiveField(next.key);
};

const assignedHeaders = new Set(Object.values(state.columns).filter(Boolean));
const fieldForHeader = (h) => FIELDS.find((f) => state.columns[f.key] === h);

async function save() {
  if (problems.length) return;
  setSaving(true);
  setServerErrors([]);
  try {
    const res = await api.saveSetup(detail.id, {
      account_number: state.columns.account_number,
      chargeoff_date: state.columns.chargeoff_date,
      client_ref: state.columns.client_ref || null,
      date_format: state.dateFormat,
      skip_footer_rows: Number(state.skipFooterRows) || 0,
```

```
      client_name: state.clientSource === "file_column"
        ? { source: "file_column", header: state.clientHeader } : { source: "as
      folders: state.folderRows.map((r) => ({ folder_path: r.folder_path, file_
                                              client_id: r.client_id })),
    });
    onSaved?.(res);
  } catch (e) {
    setServerErrors(e.errors?.length ? e.errors : [e.message]);
  } finally {
    setSaving(false);
  }
}

// Keyboard: A = save & next, 1-9 = assign Nth column to the active field.
useEffect(() => {
  const onKey = (e) => {
    if (["INPUT", "SELECT", "TEXTAREA"].includes(e.target.tagName)) return;
    if (e.key.toLowerCase() === "a" && !problems.length && !saving) { e.prevent
    const n = Number(e.key);
    if (n >= 1 && n <= 9 && sample.headers[n - 1]) assignToActive(sample.header
  };
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
});

return (
  <div className="pi-setup">
    {/* ---- Sample table: click a column header to assign it to the active fie
    <section className="pi-sample" aria-label="Sample rows (masked)">
      <p className="pi-hint">Click a column to use it for <strong>{FIELDS.find(
      <div className="pi-sample-scroll">
        <table>
          <thead>
            <tr>
              {sample.headers.map((h, i) => {
                const f = fieldForHeader(h);
                return (
                  <th key={h + i} className={f ? "is-assigned" : ""}>
                    <button type="button" onClick={() => assignToActive(h)}
                            aria-label={`Use column ${h} for ${activeField.repl
                      <span className="pi-colnum">{i + 1}</span> {h}
                      {f && <span className="pi-colfield">{f.label}</span>}
                    </button>
                  </th>
                );
              })}
```

```
        </tr>
      </thead>
      <tbody>
        {sample.rows.map((r, ri) => (
          <tr key={ri}>{r.map((v, ci) => <td key={ci} className={fieldForHe
        ))}
      </tbody>
    </table>
  </div>
  {sample.rows.length === 0 && <p className="pi-muted">No sample rows could
</section>

{/* ---- Field rows ---- */}
<section className="pi-fields">
  {FIELDS.map((f) => {
    const sug = detail.suggestion?.fields?.[f.key];
    const isSuggested = sug && sug.header === state.columns[f.key] && !touc
    return (
      <div key={f.key} className={`pi-field-row ${activeField === f.key ? "
           onFocus={() => setActiveField(f.key)} onClick={() => setActiveFi
        <label htmlFor={`pi-${f.key}`}>{f.label}{f.required && <span aria-h
        <select id={`pi-${f.key}`} value={state.columns[f.key]} onChange={(
          <option value={NOT_IN_FILE}>{f.required ? "Choose a column…" : "N
          {sample.headers.map((h) => (
            <option key={h} value={h}>{h}{assignedHeaders.has(h) && state.c
          ))}
        </select>
        {isSuggested && <ConfidenceChip conf={sug.conf} reason={sug.reason}
        {f.key === "chargeoff_date" && (
          <div className="pi-date">
            <label htmlFor="pi-datefmt">Date format</label>
            <select id="pi-datefmt" value={state.dateFormat} onChange={(e)
              <option value="">Choose…</option>
              {detail.date_formats.map((d) => <option key={d} value={d}>{d}
            </select>
            <DatePreview preview={datePreview} onUseSuggested={(lbl) => set
          </div>
        )}
      </div>
    );
  })}

  {/* Client name source */}
  <fieldset className="pi-field-row">
    <legend>Client Name *</legend>
    <label><input type="radio" checked={state.clientSource === "assigned"}
```

```
          <label><input type="radio" checked={state.clientSource === "file_column
          {state.clientSource === "file_column" && (
            <select aria-label="Client name column" value={state.clientHeader} on
              <option value="">Choose a column…</option>
              {sample.headers.map((h) => <option key={h} value={h}>{h}</option>)}
            </select>
          )}
        </fieldset>

        <details className="pi-advanced">
          <summary>Advanced</summary>
          <label>Footer / trailer lines to skip at the end <input type="number" m
          <p className="pi-muted">Delimiter: {detail.delimiter} · Columns: {detai
        </details>
      </section>

      {state.clientSource === "assigned" && (
        <FolderClientTable formatId={detail.id} rows={state.folderRows} prod={pro
                           onChange={(folderRows) => set({ folderRows })} />
      )}

      {/* ---- Footer ---- */}
      <footer className="pi-setup-footer">
        <p>This applies to all <strong>{detail.file_count}</strong> files with th
        {serverErrors.length > 0 && <ul className="pi-errors" role="alert">{serve
        <div className="pi-actions">
          <button type="button" className="btn-secondary" onClick={onSkip}>Skip f
          <span title={problems.join("\n")}>
            <button type="button" className="btn-primary" disabled={!!problems.le
              {saving ? "Saving…" : "Save & next"} <kbd>A</kbd>
            </button>
          </span>
        </div>
        {problems.length > 0 && <p className="pi-muted">To save: {problems[0]}</p
      </footer>
    </div>
  );
}

export function ConfidenceChip({ conf, reason }) {
  const label = { HIGH: "Looks right", MEDIUM: "Please check", LOW: "Best guess"
  return <span className={`pi-chip pi-conf-${(conf || "none").toLowerCase()}`} ti
}

function DatePreview({ preview, onUseSuggested, current }) {
  if (!preview) return null;
```

```
  const pct = Math.round(preview.rate * 100);
  return (
    <div className={`pi-datepreview ${pct >= 95 ? "ok" : "bad"}`} aria-live="poli
      <ul>{preview.samples.slice(0, 3).map((s, i) => <li key={i}><code>{s.raw}</c
      <span>{pct}% of sample dates read correctly</span>
      {pct < 95 && preview.suggested && preview.suggested !== current && (
        <button type="button" className="btn-link" onClick={() => onUseSuggested(
          Use {preview.suggested} instead
        </button>
      )}
    </div>
  );
}

function FolderClientTable({ formatId, rows, prod, onChange }) {
  const update = (key, patch) => onChange(rows.map((r) => (r.key === key ? { ...r
  const addSplit = (folder_path) => onChange([...rows, { key: `${folder_path}|new
                                                          files: null, file_patter
  const remove = (key) => onChange(rows.filter((r) => r.key !== key));
  const first = rows.find((r) => r.client_id);

  return (
    <section className="pi-folders">
      <header>
        <h3>Client by folder</h3>
        {rows.length > 1 && first && (
          <button type="button" className="btn-link"
                  onClick={() => onChange(rows.map((r) => ({ ...r, client_id: fir
            Use {first.client_label} for all folders
          </button>
        )}
      </header>
      <table>
        <thead><tr><th>Folder</th><th>Files</th><th>Client</th><th aria-label="ac
        <tbody>
          {rows.map((r) => (
            <tr key={r.key} className={r.files === null ? "is-split" : ""}>
              <td>
                {r.files === null ? (
                  <PatternInput formatId={formatId} folder={r.folder_path} value=
                                onChange={(file_pattern) => update(r.key, { file_
                ) : <span title={r.folder_path}>{shortPath(r.folder_path)}</span>
              </td>
              <td>{r.files ?? ""}</td>
              <td><ClientSelect prod={prod} value={r.client_id} label={r.client_l
                                onChange={(client_id, client_label) => update(r.k
```

```
              <td>
                {r.files === null
                  ? <button type="button" className="btn-link" onClick={() => rem
                  : <button type="button" className="btn-link" onClick={() => add
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </section>
  );
}

function PatternInput({ formatId, folder, value, onChange }) {
  const [result, setResult] = useState(null);
  useEffect(() => {
    if (!value.trim()) { setResult(null); return undefined; }
    const t = setTimeout(() => api.patternTest(formatId, folder, value).then(setR
    return () => clearTimeout(t);
  }, [formatId, folder, value]);
  return (
    <div className="pi-pattern">
      <span className="pi-muted">{shortPath(folder)} /</span>
      <input placeholder="e.g. ATT_PRE_*.txt" value={value} onChange={(e) => onCh
      {result && <span className={result.count ? "pi-muted" : "pi-warn"}>{result.
    </div>
  );
}

function ClientSelect({ prod, value, label, onChange }) {
  // Searchable select. CODEX: use iResolve's async combobox component if one exi
  const [q, setQ] = useState("");
  const [options, setOptions] = useState([]);
  const [open, setOpen] = useState(false);
  useEffect(() => {
    if (!open) return undefined;
    const t = setTimeout(() => api.clients(prod, q).then((d) => setOptions(d.clie
    return () => clearTimeout(t);
  }, [open, q, prod]);
  return (
    <div className="pi-combobox">
      <button type="button" className={value ? "" : "is-empty"} onClick={() => se
        {value ? label : "Choose client…"}
      </button>
      {open && (
        <div className="pi-combobox-pop" role="listbox">
```

```
          <input autoFocus placeholder="Search code or name" value={q} onChange={
          {options.map((c) => (
            <button type="button" role="option" aria-selected={c.id === value} ke
                    onClick={() => { onChange(c.id, `${c.code} — ${c.name}`); set
              <strong>{c.code}</strong> {c.name}
            </button>
          ))}
          {options.length === 0 && <p className="pi-muted">No clients match. Clie
        </div>
      )}
    </div>
  );
}

export function shortPath(p) {
  const parts = p.split(/[\\/]/).filter(Boolean);
  return parts.length > 3 ? `…\\${parts.slice(-3).join("\\")}` : p;
}
```

B.20 `frontend/placement-intake/TemplateImportReport.jsx`

> 🟠 **REFERENCE UI · NOT COMPILED**

```
// Option 2: result of an uploaded template. Fix buttons re-validate one row on t
// CODEX: keep the behaviour; restyle with iResolve components.
import { useMemo, useState } from "react";
import { api } from "./api";
import { shortPath } from "./FileTypeSetup";

const STATUS = {
  ready: { icon: "●", label: "Ready", cls: "ok" },
  update: { icon: "●", label: "Will update", cls: "warn" },
  error: { icon: "●", label: "Needs fixing", cls: "bad" },
};

export default function TemplateImportReport({ initial, onCommitted, onCancel })
  const [imp, setImp] = useState(initial);
  const [filter, setFilter] = useState(initial.counts.error ? "error" : "all");
  const [busyRow, setBusyRow] = useState(null);
  const [committing, setCommitting] = useState(false);
  const [confirmOpen, setConfirmOpen] = useState(false);
  const [error, setError] = useState(null);

  const rows = useMemo(() => (filter === "all" ? imp.rows : imp.rows.filter((r) =
  const valid = imp.counts.ready + imp.counts.update;
```

```
async function applyFix(row, fix) {
  setBusyRow(row.row_id);
  setError(null);
  try {
    setImp(await api.importFix(imp.id, row.row_id, fix.field, fix.value));
  } catch (e) { setError(e.message); } finally { setBusyRow(null); }
}

async function commit() {
  setCommitting(true);
  try {
    const res = await api.importCommit(imp.id);
    onCommitted?.(res);
  } catch (e) { setError(e.message); setCommitting(false); setConfirmOpen(false
}

return (
  <div className="pi-import">
    <header className="pi-import-summary">
      <h2>Checked: {imp.file_name}</h2>
      <div className="pi-tabs" role="tablist">
        {[["all", `All ${imp.rows.length}`], ["ready", `Ready ${imp.counts.read
          ["update", `Will update ${imp.counts.update}`], ["error", `Needs fixi
          <button key={k} role="tab" aria-selected={filter === k} className={fi
                  onClick={() => setFilter(k)}>{l}</button>
        ))}
      </div>
    </header>

    {error && <div className="pi-banner bad" role="alert">{error}</div>}

    <table className="pi-table">
      <thead><tr><th>Row</th><th>Folder / file name</th><th>Client</th><th>File
      <tbody>
        {rows.map((r) => (
          <tr key={r.row_id} className={`is-${STATUS[r.status]?.cls}`}>
            <td>{r.row_id}</td>
            <td><span title={r.folder_path}>{shortPath(r.folder_path || "")}</s
            <td>{r.client || "—"}</td>
            <td>{r.matching_files ?? "—"}</td>
            <td><span className={`pi-chip ${STATUS[r.status]?.cls}`}>{STATUS[r.
            <td>
              {r.messages.length === 0 && r.status === "update" && <span classN
              {r.messages.map((m) => <div key={m}>{m}</div>)}
              {r.fixes.map((fx) => (
```

```
                  <button key={fx.field + fx.value} type="button" className="btn-
                          onClick={() => applyFix(r, fx)}>{busyRow === r.row_id ?
                ))}
              </td>
            </tr>
          ))}
          {rows.length === 0 && <tr><td colSpan={6} className="pi-empty">Nothing
        </tbody>
      </table>

      <footer className="pi-sticky-footer">
        {imp.counts.error > 0 && (
          <a className="btn-secondary" href={api.importErrorsUrl(imp.id)}>Downloa
        )}
        <button type="button" className="btn-secondary" onClick={() => api.import
        <button type="button" className="btn-primary" disabled={!valid || committ
          Save {valid} file type{valid === 1 ? "" : "s"}
        </button>
      </footer>

      {confirmOpen && (
        <div className="pi-dialog" role="dialog" aria-modal="true" aria-labelledb
          <h3 id="pi-confirm-title">Save {valid} file types?</h3>
          <p>{imp.counts.ready} new, {imp.counts.update} replacing an existing se
             {imp.counts.error > 0 && ` ${imp.counts.error} rows with problems wi
          <p>Files already waiting in these folders will be processed straight af
          <div className="pi-actions">
            <button type="button" className="btn-secondary" onClick={() => setCon
            <button type="button" className="btn-primary" onClick={commit} disabl
          </div>
        </div>
      )}
    </div>
  );
}
```